What a Loop Buys: Parameter Efficiency on Text, Extendable Depth on a Serial Task*

Qirun Li *qli858246@gmail.com*
Independent researcher

Abstract

A looped transformer runs the same block of layers several times, adding depth without adding parameters. Two benefits are claimed: it behaves like a larger model, and more loops at inference solve harder problems. We test both on web text, with a pre-registered grid of 111 language models plus follow-up runs, and on a synthetic task that needs serial steps.

On text, none of the four pre-registered hypotheses held as stated; five model sizes were too few to fit the pre-registered scaling law. In exploratory comparisons with a standard (dense) model of the same size, looping the middle four of eight layers four times gives no gain at 10M–20M parameters with up to 20 tokens per parameter. At 40M–160M with 20–40 tokens per parameter, the looped model matches a dense model about 1.3–1.5 times its size, and at most sizes the factor grows with data. At 40M and the grid's learning rate, a dense model of about the same size spread over 18 layers recovers about half of this gain; its own gain does not grow with data. At equal training compute and a shared learning rate, the dense model wins every measured comparison. At equal loss, the loop saves at most 19% of weight memory but needs 2.2–2.5 times the KV cache and decoding time. Extra loops at inference never help on text.

On the synthetic task, chained lookups in random tables, looped models trained on 1–8 lookups solve longer chains when run with more loops, while dense models of equal depth and models given pause tokens do not: with 16 loops, an 8-loop model goes from 0.04 to 0.83 accuracy on 12-step chains, and in that model linear probes decode one more intermediate value after each loop. A chain of thought trained on every intermediate value is perfect within the training range at under a third of the 8-loop model's compute; beyond it, it fails from its first tokens, likely because the prompts are longer than any seen in training.

1 Introduction

A looped transformer, also called a recurrent-depth transformer, runs one block of layers several times in a row (Dehghani et al., 2019; Geiping et al., 2025; Zhu et al., 2025). Running a block r times costs r times its compute but adds almost no parameters. Two benefits are expected in return. The first is *parameter efficiency*: a looped model should behave like a larger *dense* model, a standard transformer that runs each of its layers once. This matters wherever memory or bandwidth, rather than compute, limits deployment. The second is *latent reasoning*: if each pass through the loop carries the computation one step further, running more loops at inference should solve harder problems without writing out a chain of thought. The loop would then act as test-time compute inside the network.

Both claims are hard to check from the literature. Studies differ in data, tokeniser, training recipe and model family, and seeds are rarely reported. For the first claim, the quantity of interest, how much one more loop is worth, is usually read off a single fitted scaling law (Schwethelm et al., 2026; Prairie et al., 2026; Chen et al., 2026a). For the second, the evidence comes from two lines of work that are seldom compared. Large

*This study was carried out with the help of an AI assistant (Claude, by Anthropic); Section 8 states what it did and what the author decided.

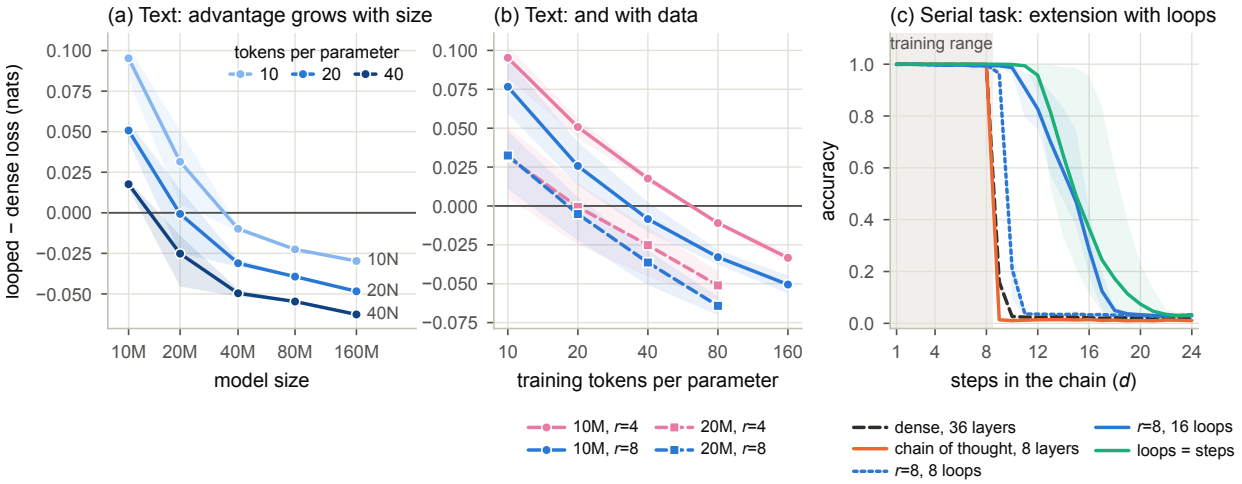


Figure 1: The main results. **(a)** Web text, at equal parameters and training tokens: looped minus dense loss for the model whose middle block is looped 4 times (lower is better for the loop: below zero, the looped model has the lower loss), at 10, 20 and 40 training tokens per parameter (10N–40N; band: range over seeds). It is worse than the dense model at 10M, mixed at 20M and better from 40M on. **(b)** The advantage grows with the number of training tokens per parameter: the curves fall further below zero (10M and 20M, 4 and 8 loops). **(c)** The chain task, trained on chains of 1–8 lookups (shaded). Run with more loops, looped models solve longer chains: "r=8, 16 loops" is trained with 8 loops and run with 16, and "loops = steps" is trained and run with one loop per step of the chain. Dense models fail beyond the training range, including one with the executed depth of 8 loops (36 layers) and one that writes the chain out. The 8-loop model run at its training loop count ("r=8, 8 loops") gets one step past the range and then fails. Mean over five seeds; band: range.

models trained with a random number of loops improve on reasoning benchmarks as more loops are run, up to a task-dependent limit (Geiping et al., 2025). Small models on synthetic tasks solve longer problems when run longer (Fan et al., 2025; Kohli et al., 2026; Zhang et al., 2026).

We test both claims with one model family and one training recipe. On web text, a pre-registered grid of 111 language models varies the model size (looped models of 10M–80M parameters, dense models up to 160M), which layers are looped, the loop count r and how gradients flow through the loop. In this grid, a dense model runs eight layers once. A looped model runs either all eight layers r times or only the middle four, with two layers before the loop and two after it (Figure 2a). With the middle block looped four times, it runs 20 layers in total; most of our text results are for this 4-loop model. Follow-up runs add a 160M looped model, more training data, re-tuned learning rates and deeper dense models (Section 3.2).

On web text at these sizes, extra loops at inference did not help (Section 5). We therefore also built a synthetic task that needs serial steps, and fixed its design and predictions before the main runs. Each problem is a chain of lookups in fixed random tables, one lookup per step; we call it the chain task. On it we compare looped models with dense models of the same size, dense models that run as many layers in total (the same *executed depth*), dense models that write out a chain of thought, and dense models given pause tokens before the answer.

Earlier measurements of what a loop is worth on text do not vary model size and training tokens per parameter separately with several seeds (Section 2). Doing so here shows that the value of a loop depends on both, and a dense control of nearly the same size and depth shows how much of it is depth. On the chain task, we replicate the finding that looped models solve longer chains when run with more loops (Kohli et al., 2026; Zhang et al., 2026), and add probes of every intermediate value, comparisons on the same problems with a written chain of thought and with pause tokens, and stopping rules that do not use the answer.

Table 1: The two claims on the two tasks, by what the looped model is compared with. Text entries are for the 4-loop model, except in the equal-compute row, which covers every looped model whose dense comparison was measured rather than extrapolated (258 comparisons, each one looped result against the dense model trained with the same compute). Looped models run at their training loop count unless stated. Steps solved: the longest chain length up to which every length is solved with accuracy of at least 0.9, computed per seed and averaged over five seeds.

Compared with	Web text, 10M–160M	Chain task, 6.6M, trained on 1–8 steps
Claim one: a looped model behaves like a larger one		
Dense, equal parameters and data	Worse at 10M, mixed at 20M, better from 40M on (10–40 tokens per parameter). At 40M–160M with 20–40 tokens per parameter, as good as a dense model 1.3–1.5 times its size, more with more data (§4.2)	Better: 8.0 steps solved with 4 loops, against 4.2 (§6.2)
Dense, equal parameters, nearly equal depth (18 against 20 layers)	At 40M, recovers about half of the advantage, but not the part that grows with data. With its own learning rate re-tuned, it recovers more at the shortest budget (one seed) (§4.3)	Not measured
Dense, equal training compute	Dense better in all 258 measured comparisons at the shared learning rate; after retuning the learning rates, one comparison is a tie (§4.4)	Loop at least as good as dense models of equal executed depth, which here cost about the same compute and have 2.4–4.4 times its parameters: 4 and 8 loops solve 8.0 and 9.0 steps, dense models of 20 and 36 layers 6.4 and 8.0 (§6.2)
Written chain of thought	Not measured	Chain of thought better within the range: perfect, at under a third of the 8-loop model's inference compute, but supervised on every intermediate value (§6.2)
Deployment, equal loss	4–19% less weight memory (19% against an extrapolated dense size), but 2.2–2.5 times the KV cache and decoding time (§4.5)	Not measured
Claim two: more loops at inference solve harder problems		
Its own training loop count	Never better (§5)	Longer chains: the 8-loop model goes from 0.04 to 0.83 at 12 steps with 16 loops. Most models can decide when to stop from their own confidence (exploratory; §6.3, §6.6)
Other extra computation	Not measured	Dense models of equal depth and pause tokens do not solve chains beyond the training range; nor does a chain of thought, which fails from its first tokens on the longer inputs (§6.3)

Results that were planned before their runs are marked as such; all other analyses are labelled exploratory (Section 3.4).

Figure 1 and Table 1 summarise the results. On web text, the pre-registered scaling-law analysis failed. Five model sizes were too few to fit the law it relied on, and none of its hypotheses about where to loop, how often and how to backpropagate held as stated (Section 4.1). The other text results therefore come from direct comparisons that need no fitted law.

At equal parameters, the value of the loop depends on model size and data. At 10M–20M with up to 20 training tokens per parameter, the 4-loop model is no better than a dense model of its own size. At about 40M–160M with 20–40 tokens per parameter, it is as good as a dense model 1.3–1.5 times its size, and the factor grows with the number of training tokens per parameter (Section 4.2). Part of this advantage comes from depth. At 40M, a deeper dense model of the same size, with 18 layers, recovers about half of it. The deeper model's own value does not grow with data at 40M, so depth does not explain why the loop's advantage does. With a tuned learning rate, the deeper model recovers more at the shortest training budget (one seed; Section 4.3).

At equal training compute, the dense model wins every measured comparison at a shared learning rate; after re-tuning the learning rates, one comparison is a tie (Section 4.4). At deployment, the loop saves some weight memory, but its decoding time and KV cache grow with every layer it runs (Section 4.5). Extra loops at inference never help on text. Depending on how it is trained, the looped block learns a fixed-length program that works only at its training loop count, a cycle that repeats every few loops, or a fixed point where the state stops changing (Section 5).

The chain task (Section 6) gives the opposite picture. In models with at least one loop per step, linear probes decode one more intermediate value after each loop. Within the training range, a written chain of thought solves the task more cheaply. Beyond it, looped models run with more loops solve longer chains, while dense models of equal executed depth and models with pause tokens do not. The chain of thought fails there too, from its first written tokens. Since longer chains here also mean longer prompts, this may be a failure to generalise to longer inputs rather than to more steps, and only a task with inputs of fixed length could tell. Most looped models can also decide when to stop: stopping once the top probability passes a threshold works well without knowing the correct answer, although we chose this rule only after comparing several (Section 6.6).

We read the difference between the two settings as follows. Extra loops help when the model's errors come from too few serial steps and the loop can run longer without destroying its state; using them in deployment also needs a signal for when to stop. The chain task meets these conditions once the model is trained for them; on web text at these sizes we saw no sign of the first. We have not tested this interpretation, and Section 7 says what would. Section 8 covers the limitations, the pre-registration and the released code and results.

2 Related work

Claim one: what a loop is worth. Sharing parameters across layers trades compute for parameters (Dehghani et al., 2019; Lan et al., 2019; Bae et al., 2025a). Transformers that re-use parameters "perform slightly better when comparing models with equal parameter count, but slightly worse when accounting for reuse and comparing per FLOP" (Kaplan et al., 2020, App. D.2, Fig. 17).

The law we test comes from Schwethelm et al. (2026): $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$, where N_{once} counts the parameters run once and N_{rec} those in the loop. With $\varphi = 0$ the looped parameters count once, as in the model's actual size; with $\varphi = 1$ they count r times, as in the unrolled network. Schwethelm et al. estimate $\varphi = 0.46$ on FineWeb-Edu, and we follow their choice of two unshared layers before the loop and two after it. This layout of a prelude (two layers), a core (four layers) and a coda (two layers), and the input injection described in Section 3.1, follow Huginn (Geiping et al., 2025). Schwethelm et al. also find that backpropagating through only the last $\lceil r/2 \rceil$ loops, as Prairie et al. (2026) do, lowers φ from 0.45 (their baseline refitted on the same budgets) to 0.38. That comparison spends the saved FLOPs on more tokens. Our first hypothesis (H1, Section 3.4) tests truncation at equal tokens and at equal FLOPs separately.

Other studies measure the value of a loop in different settings, with mixed results. Parcae (Prairie et al., 2026) finds that looping lowers perplexity at matched parameters and tokens, and that at a fixed FLOP budget the amount of looping should grow with the amount of data. Looped models without sparse layers scale worse than non-looped ones at equal FLOPs (Lee et al., 2026). With repeated data, the best loop count grows with compute (Chen et al., 2026b), and a bounded function of r extrapolates better than r^{φ} (Chen et al., 2026a). Looping the middle half of the layers beats looping all of them by less than 0.01 nats in single-seed ablations (Wang et al., 2026b). A dense model of the same depth beats the looped one in a controlled appendix of Ouro (Zhu et al., 2025), and plain recursion is worse than a vanilla model in validation loss at matched compute (Bae et al., 2025b). A 1.4B looped model beats a dense model of its own size on all 12 of its benchmarks, but not a 3.9B dense model of matched inference compute (Marchenko et al., 2026). In data-limited pretraining, looped transformers beat standard ones on downstream evaluations at 10M and 100M words (Gülbahar et al., 2026). None of these studies varies model size and tokens per parameter separately with several seeds, which is what our grid adds.

Deployment. Without sharing, the KV cache and the decoding time grow with the loop count (Wu et al., 2025; Monea et al., 2026). Ouro shares the cache during decoding only, and full backpropagation through a fixed depth prevents sharing the cache across loops in Huang et al. (2026). Models pretrained so that only the first loop writes a cache lose no quality (Monea et al., 2026).

Claim two: loops as test-time compute. Recurrent networks solve harder instances by running more iterations, and they can "overthink" (Schwarzschild et al., 2021; Bansal et al., 2022). Looped transformers with an input-dependent number of loops generalise to longer inputs on algorithmic tasks (Fan et al., 2025). Recurrent layers with input injection, on top of position embeddings made for the task, help arithmetic generalise further to longer numbers (McLeish et al., 2024). Recurrent-depth transformers extrapolate to deeper chains of composed facts with more loops, while vanilla transformers of equal parameters or FLOPs do not (Kohli et al., 2026). The number of input positions a loop resolves per iteration scales with the training length divided by the training loop count (Zhang et al., 2026). Shallow transformers can also learn shortcuts for associative serial computations (Liu et al., 2022).

What a loop converges to matters. Looped and equilibrium models settle into fixed points or orbits (Geiping et al., 2025; Logan, 2026; Bai et al., 2019). Those whose fixed point does not depend on the starting state use extra test-time computation better (Anil et al., 2022), and looping gains concentrate on reasoning tasks (Saunshi et al., 2025). We read the state after every loop the way the logit lens (nostalgebraist, 2020) and the tuned lens (Belrose et al., 2023) read layers.

A chain of thought (Wei et al., 2022) adds serial computation to a fixed-depth transformer (Merrill & Sabharwal, 2024). We train it from scratch on the written values, as in scratchpad training (Nye et al., 2021), and compare it with loops on the same problems, together with pause tokens (Goyal et al., 2024). Filler tokens help on some algorithmic tasks (Pfau et al., 2024). RELAY aligns loop iterations with chain-of-thought steps (Yu et al., 2025), and continuous thoughts feed the hidden state back as input (Hao et al., 2025). Transformers do not generalise robustly to inputs longer than those seen in training (Zhou et al., 2024; Cho et al., 2024).

Adaptive computation time (Graves, 2016), PonderNet (Banino et al., 2021) and Mixture-of-Recursions (Bae et al., 2025b) learn when to halt during training, and Fan et al. (2025) take the loop count at which the model is most confident. We instead compare stopping rules that do not use the answer, applied after training and fixed in advance.

Our chain-task results replicate, with five seeds, that looped models solve longer chains with more loops (Kohli et al., 2026; Zhang et al., 2026). We add probes of every intermediate value, the effect of the learning rate, the stopping rules, and comparisons on the same problems with a written chain of thought and with pause tokens.

3 Setup

3.1 Models

All language models of the grid (Figure 2a) are decoder-only transformers with eight unique layers of width w. They use pre-norm RMSNorm, SwiGLU feed-forward blocks (hidden size $\frac{8}{3}w$), rotary position embeddings, attention heads of dimension 64, no biases, untied input and output embeddings, a 16k BPE vocabulary trained on the training data, and a context of 1024 tokens.

The models differ only in how they run these layers. A *dense* model runs its eight layers once. A *middle-block* looped model runs the first two layers once (the prelude), the middle four r times (the core, or looped block) and the last two once (the coda), so it executes $2 + 4r + 2$ layers; a *whole-stack* looped model runs all eight layers r times.

Each loop starts from a linear map of the previous state concatenated with the prelude output (for the whole stack, which has no prelude, the embedded input). This input injection, as in Huginn (Geiping et al., 2025), has $2w^2$ parameters; the first state is zero. Every residual branch is scaled by $(L/12)^{-1/2}$, a depth rule in which $L = 8$ is the number of unique layers and 12 the reference depth, and by a further $1/r$ inside the

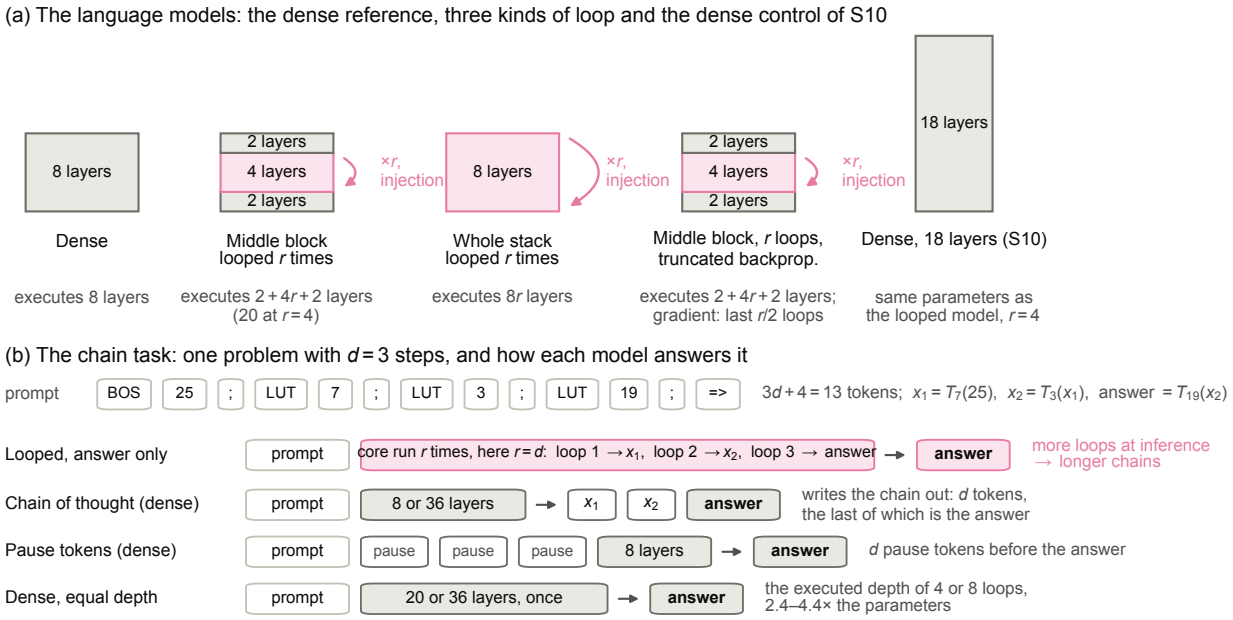


Figure 2: The models and the chain task. **(a)** Every language model of the pre-registered grid has eight unique layers. A middle-block looped model runs its middle four layers r times; a whole-stack model runs all eight. Each loop starts from a linear map of the previous state concatenated with the prelude output (input injection). With truncated backpropagation, only the last $r/2$ loops receive gradients. S10 adds dense models with 18 layers and about the looped model's number of parameters (Section 4.3). **(b)** A chain-task problem with $d = 3$ lookups in fixed random tables T_k (`LUT k`: look up table k; BOS: the start marker). The looped model answers in one forward pass and computes the chain inside the loop (drawn with one loop per step); run for more loops, it solves longer chains. A dense chain-of-thought model writes the d values of the chain, the last of which is the answer. A dense model with d pause tokens and a dense model of the looped model's executed depth answer in one pass.

core. The scaling is meant to keep the best learning rate independent of r (Wang et al., 2026a); Appendix B checks this. Backpropagation runs through all loops or, in *truncated* runs, through the last $r/2$ loops only. A model's *executed depth* is the number of layers it runs: 8 for the dense model and $2 + 4r + 2$ for the middle block. Most text results are for *the 4-loop model*: the middle block looped 4 times with full backpropagation, which runs 20 layers.

There are five sizes, named by their approximate non-embedding parameters: 10M, 20M, 40M, 80M and 160M, with widths 320 to 1280. Their parameter counts are $N = 15.4$M, 26.8M, 50.1M, 92.7M and 179.6M. N counts the non-embedding parameters and also the output head (Porian et al., 2024), which is large at small widths with a 16k vocabulary (Table 6 in Appendix B).

3.2 Data, training and the grid

Every run, except those of one supplementary experiment that varies the data order (S7), reads a prefix of one fixed-order stream: files 000–012 of FineWeb-Edu sample-10BT (Penedo et al., 2024). The stream has 10.4B tokens with our tokeniser and the longest run uses 9.9B, so no run sees a document twice. The main validation set is 50M held-out FineWeb-Edu tokens from file 013. Three further sets of 20M tokens come from SlimPajama (Soboleva et al., 2023), FineMath (Ben Allal et al., 2025) and GitHub code (Appendix B).

We use AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0) with one learning rate per width, chosen by a sweep on the dense models and used unchanged for the looped models. This rate applies to the embedding and output layers; the hidden layers use $(8/12)^{-1/2}$ times it, by the same depth rule as the

residual scaling (Appendix B). A budget is the number of training tokens per parameter, written $10N$ for ten tokens per parameter. The schedule is warm-up–stable–decay with cooldown branches (Hägele et al., 2024): each run has one constant-rate trunk, and for each budget a cooldown branches off it at 80% of the budget and decays the rate linearly to zero at the budget. Each run therefore gives one result per budget.

Looped runs have budgets of $10N$, $20N$ and $40N$; dense runs $5N$ to $80N$ ($40N$ at 160M). Each looped size (10M–80M) has ten cells: the middle block with $r = 2$, 4 and 8 and full backpropagation, and with $r = 4$ and 8 and truncated backpropagation; and the same five for the whole stack. Each size also has one dense model. Every looped cell and dense model has three seeds at 10M–40M and one at 80M, and a 160M dense model serves as a reference: 100 looped and 11 dense runs, 111 in the pre-registered grid. Seeds change the initialisation, not the data order.

Extensions of the grid, reported as exploratory, add a 160M 4-loop model and a second 160M dense seed, and continue 10M models to $160N$ and 20M models to $80N$ (Appendix B lists the cells and seeds). They also train 20M models with a random loop count, a regime the pre-registration declared out of scope. Ten supplementary experiments, S1–S10, ran under plans frozen before their runs (Table 7 in Appendix B). Everything ran on four RTX 4090 GPUs, about 19 days of wall-clock time in all.

3.3 Measurement

Losses are in nats per token on the main validation set. Across initialisation seeds, the loss of the 20M dense model spans 0.014–0.024 nats at $10N$–$40N$ (0.004–0.008 at 10M and 40M; 0.003–0.07 across the looped cells at 20M). Pairing runs by seed does not remove this noise. Runs with the same seed and width start from the same weights in the layers they share and read the same data, yet the gap between the 4-loop model and the dense model of the same seed varies across seeds by 0.005–0.020 nats at 10M, 0.032–0.045 at 20M and 0.003–0.005 at 40M. A difference of 0.01 nats in one seed is therefore within the noise at 10M–20M and resolvable at 40M. The grid's results at 10M–40M are means over three seeds.

The pre-registered analysis measures the value of a loop through the effective size of a looped model, $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$ (Schwethelm et al., 2026), where N_{once} counts the parameters run once and N_{rec} those in the loop. Unlike Schwethelm et al. (2026), who count non-embedding parameters only, we include the output head in N_{once}, as in N. A dense "ruler" $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$ is fitted to the dense runs (Hoffmann et al., 2022; Besiroglu et al., 2024) with a Huber loss in log space and block-bootstrap intervals (Appendix C). With the ruler fixed, the primary analysis fits one φ for each loop type, that is, each combination of looped block and backpropagation (the four rows of Table 2), so that the ruler evaluated at N_{eff} matches the looped losses. A joint fit of the ruler and all φ is the pre-registered secondary analysis.

Three further measures need no fitted law. The *gap* is the looped minus the dense loss at equal parameters and tokens, with the same seed. The *dense size of equal loss*, $N^{\star}$, is the size of a dense model that would reach the looped model's loss on the same number of training tokens. We read each measured dense size's loss at that token count and interpolate $\log N$ linearly in the loss. The *multiplier* $M = N^{\star}/N$ says how many dense parameters each parameter of the looped model is worth; $M > 1$ means the looped model beats a dense model of its own size. A local exponent φ_{loc} is defined by $N^{\star} = N_{\text{once}} + r^{\varphi_{\text{loc}}} N_{\text{rec}}$. Finally, the *equal-compute gap* is the looped loss minus the loss of a dense model of the same size trained with the same FLOPs. We convert a looped run's training FLOPs (counted over executed layers, including attention and the output head) into the tokens a dense model of the same size could have seen with them. We then read the dense loss at that number of tokens off the measured dense curve. Where that point lies beyond the measured curve, two extrapolations bound it (Section 4.4).

3.4 Hypotheses and evidence labels

The pre-registration was frozen on 2026-09-23, before any training run. It states four hypotheses with decision rules (Appendix A):

- **H1**: truncated backpropagation lowers φ, under both an equal-token and an equal-FLOP accounting.
- **H2**: a looped middle block has a higher φ than a looped whole stack.

Table 2: Pre-registered results. Top: φ per loop type on the main validation set (equal-token accounting, N including the output head), from the primary analysis with 95% block-bootstrap intervals and from the secondary joint fit; both fits bound φ below by 0. Bottom: the verdicts on the four hypotheses under the frozen decision rules (in full in Table 5).

Loop type	φ, primary [95% interval]	φ, joint fit
Middle block, full backpropagation	0.13 [0.02, 0.27]	0.41
Middle block, truncated	0.00 [0.00, 0.17]	0.29
Whole stack, full backpropagation	0.00 [0.00, 0.16]	0.00
Whole stack, truncated	0.17 [0.10, 0.20]	0.29

Hypothesis	Verdict
H1: truncation lowers φ	Not supported
H2: a middle block has a higher φ than the whole stack	Depends on the measure
H3: the gain from loops saturates in r	Not supported
H4: the best r is small at equal training compute, larger at equal deployment compute	First half trivially (dense is best); second not supported

- **H3**: the gain from extra loops saturates in r, so that a geometric form $N_{\text{eff}} = N_{\text{once}} + N_{\text{rec}}(1-\rho^r)/(1-\rho)$, with a fitted ratio ρ, extrapolates better than r^φ to a held-out size.
- **H4**: the compute-optimal r is small at matched training FLOPs and larger at matched deployment FLOPs.

Each result carries one of three labels. *Pre-registered* results test a hypothesis and decision rule frozen before training. *Pre-specified* results were written into a plan frozen before the runs they describe: the synthetic design (2026-10-09) and the three supplementary plans (2026-10-13, 2026-10-15 and 2026-10-27). These plans call their experiments exploratory because they lie outside the pre-registration, but their quantities and readings were fixed before the runs. *Exploratory* results were chosen after seeing data. Every departure from a frozen document is dated in the deviation log.

4 What a loop is worth on text

4.1 The pre-registered law cannot say what a loop is worth

The pre-registration measured the value of a loop by one number per loop type: the exponent φ of $N_{\text{eff}} = N_{\text{once}} + r^\varphi N_{\text{rec}}$, read off a dense ruler fitted to the grid's 54 dense results (one result per run and budget). Five model sizes were too few for this. The ruler puts its irreducible loss at the lowest value the fit allowed ($E = 0.050$) and its parameter exponent at $\alpha = 0.054$, far below the usual value of about 0.3, because five sizes cannot pin down E, A and α separately. φ inherits this weakness: it is 0–0.17 per loop type with the primary fit, and 0–0.41 with the joint fit over all 352 results (two of the grid's results are excluded; Appendix A), which gives $\alpha = 0.267$ (Table 2).

Under the pre-registered decision rules, none of the four hypotheses held as stated. Table 2 gives the verdicts and Appendix A the details. For example, truncated backpropagation was predicted to lower φ (H1). With truncation, φ was lower in the middle block, though the intervals overlap, and higher in the whole stack, where truncation also lowered the loss of the whole stack looped 4 times at every size. Truncated runs also had most of the loss spikes (Appendix B).

Fitted separately at each size against the same ruler, and allowed to be negative, φ of the 4-loop model is -0.32, 0.19, 0.39, 0.52 and 0.65 from 10M to 160M *(exploratory)*. These values come from the same weak ruler, so we use only their direction: the value of a loop rises with model size, and a single φ per loop type averages over very different regimes. The rest of this section therefore measures the value of a loop without a fitted law.

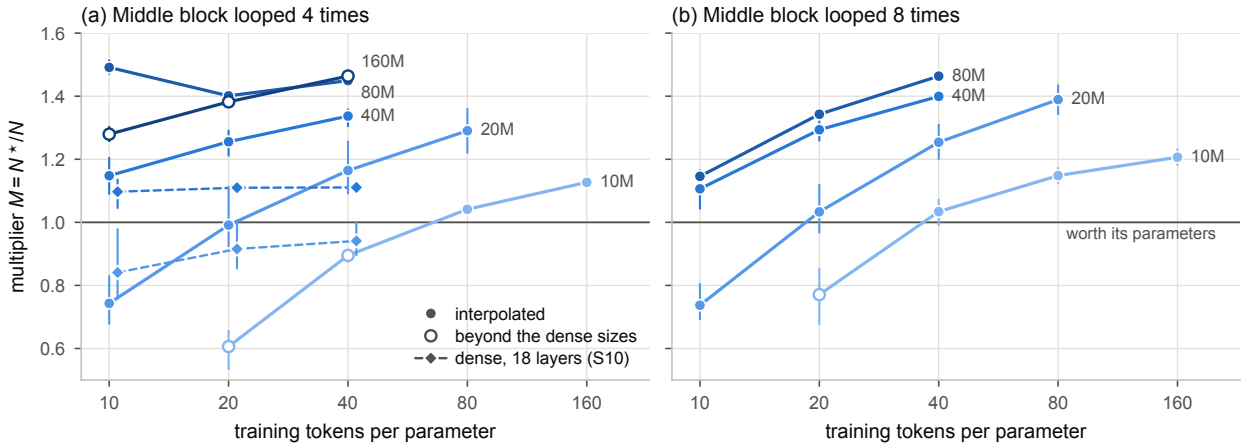


Figure 3: What each parameter of a middle-block looped model is worth in dense parameters: the multiplier $M = N^\star/N$. $N^\star$ is the size of the dense model that reaches the same loss on the same number of training tokens, interpolated between the measured dense sizes (including the dense models trained longer in S4, Section 4.4). $M = 1$: as good as a dense model of its own size. Open markers: $N^\star$ lies beyond the largest or smallest dense size, by at most a factor of two. Diamonds: dense models with 18 layers and about the looped model's number of parameters (Section 4.3). Bars: 95% intervals from resampling the seeds (three; two at 160M; one at 80M), a rough guide with so few seeds.

4.2 Measured directly, a loop's value rises with model size and data

The dense runs themselves can serve as the ruler. For each looped model, we read the loss of every measured dense size at the looped model's number of training tokens, and interpolate the dense size $N^\star$ that has the same loss. The multiplier $M = N^\star/N$ is then what each parameter of the looped model is worth in dense parameters (Figure 3, *exploratory*).

For the 4-loop model, M depends on size and data. At 10M–20M with 10–20 tokens per parameter, M is 0.6–1.0 where it can be read: the looped model is worth at most a dense model of its own size. At 40M–160M with 20–40 tokens per parameter, M is 1.26–1.46. For some of these values, $N^\star$ lies outside the measured dense sizes, by less than a factor of two: below the smallest for the 10M value of 0.6, and above the largest for the 160M values. At 10M with 10 tokens per parameter, M is not reported: the looped model is worse than both dense models measured at that number of tokens, and these two are out of order (the 20M one, trained on 5.7 tokens per parameter, is worse than the 10M one), so the dense size of equal loss cannot be extrapolated (Appendix C).

M rises with data at 10M–40M and at 160M. At 80M, with one seed, it is flat: 1.49, 1.40 and 1.45 at $10N$, $20N$ and $40N$. Fitting $\log_2 M$ as a plane in $\log_2 N$ and $\log_2$(tokens per parameter) gives a factor of 1.21 per doubling of model size and 1.14 per doubling of tokens per parameter (Appendix C). Looping 8 times instead of 4 raises M by up to 0.16 at 10M–20M (0.11 among the values within the measured dense sizes), with no consistent change from 40M on.

M counts all of N, including the output head, which lies outside the looped block (34% of N at 10M, 21% at 40M). The local exponent φ_{loc} credits the whole gain to the looped parameters; it is negative where $M < 1$ (-0.84 at 20M with 10 tokens per parameter) and 0.22–0.55 from 40M on. If N excludes the head, as in Schwethelm et al. (2026), φ_{loc} is 0.32–0.50 at 40M–160M with 20–40 tokens per parameter, close to their $\varphi = 0.46$ (Appendix C).

The shared learning rate does not create this trend. A supplementary experiment (S1, *pre-specified*; Appendix D) re-tuned the learning rates of the dense model and the 4-loop model at 10M–40M, with one seed (two at 20M). Re-tuning changed the gap between them by at most 0.005 nats at 20M–40M. At 10M, where the looped model preferred lower rates, it improved the looped model's standing by about 0.02 nats. With

each model at its best rate, the size trend at $40N$ is somewhat smaller but meets the plan's criterion, which asks that it keep more than half of its value at the shared rate. The data trend at 10M is larger than at the shared rate: the gap goes from $+0.082$ at $10N$ to -0.059 at $160N$.

Converted to M, the equal-parameter results of Parcae (Prairie et al., 2026) and Ouro (Zhu et al., 2025) reach a similar level (1.23–1.46 at 134M–770M), but not the same trend: Parcae's values fall with size at about 80 tokens per parameter, and Ouro's rise with size while its tokens per parameter fall. The shallower looped blocks of Saunshi et al. (2025), trained on about 750–1500 tokens per parameter, reach 1.47–1.95 (Appendix C). The published results barely sample models below 40M trained on 20 or fewer tokens per parameter, where we find the looped model worth at most a dense model of its own size. A value that rises with tokens per parameter is not captured by effective-parameter laws in which the factor applied to the looped parameters does not depend on the training tokens (Schwethelm et al., 2026; Chen et al., 2026a).

Every dense model of this ruler has eight layers, while the 4-loop model executes 20. At a fixed size, deeper and thinner models do better below a billion parameters (Liu et al., 2024), though the shape matters little over a wide range of shapes (Kaplan et al., 2020). So M may measure what depth gives at a fixed size rather than what sharing gives.

4.3 At 40M, a deeper dense model of the same size recovers about half of the loop's advantage

To separate depth from sharing, a supplementary experiment (S10, *pre-specified*) trained dense models with 18 layers and about the looped model's number of parameters: 18 layers of width 320 ($N = 28.1$M) at 20M and of width 448 ($N = 51.2$M) at 40M. Their executed depth of 18 is close to the 4-loop model's 20. They use the grid's recipe, data, budgets and three seeds. Their learning rate follows the grid's rule (Section 3.2): the embedding and output layers use the rate of their width, and the hidden layers use $(18/12)^{-1/2}$ times it. We call this the rule's rate. Their multiplier M_{deep} is read off the same 8-layer dense runs and compared with the 4-loop model's multiplier M_{loop} at the same size and budget. The plan fixed in advance how to interpret them. It defines $R = \log M_{\text{deep}} / \log M_{\text{loop}}$, computed at 40M for each budget and averaged over the three budgets, as the share of the looped model's advantage that a dense model of the same size and nearly the same depth recovers. The plan reads $R \geq 0.75$ as "most of the advantage is a depth effect", $0.25 \leq R < 0.75$ as "partly depth, partly sharing" and $R < 0.25$ as "looping buys more than depth".

At 40M, the deep dense model is worth 1.10, 1.11 and 1.11 dense parameters per parameter at $10N$, $20N$ and $40N$; the looped model is worth 1.15, 1.26 and 1.34 (Figure 3a, diamonds). So R is 0.50 (0.41–0.62 when the three seeds are resampled), in the band the plan reads as "partly depth, partly sharing", as it predicted. At 40M, the value of depth does not grow with data, but the looped model's lead over the deep model does. The lead is 0.003 nats at $10N$, 0.015 at $20N$ and 0.029 at $40N$, with the same sign in all three seeds at the two longer budgets. At 20M, the deep model is narrow (width 320) and worse than the 8-layer model of the same size ($M_{\text{deep}} = 0.84$–0.94). There, depth explains none of the looped model's advantage at $40N$ ($M_{\text{loop}} = 1.16$; the plan reports 20M descriptively).

The deep model's learning rate was checked with one seed (seed 42). At 40M its best rate was the highest one tried, 1.41 times the rule's rate, at all three budgets, beating the rule's rate by 0.014, 0.006 and 0.001 nats at $10N$, $20N$ and $40N$. Because it beat the rule's rate by more than 0.01 nats at $10N$, the plan asks for a second computation with each model at its best tested rate (Appendix E). Computed that way, the deep model is ahead of the looped one at $10N$ by 0.010 nats ($M = 1.37$ against 1.18, so $R = 1.91$), and R is 0.56 and 0.34 at $20N$ and $40N$ (0.45 and 0.37 for the same seed at the rule's rate). These readings may understate the deep model, because its best rate lies at the edge of the range tried and the plan had no rule to extend the range. Their average, 0.94, would count as "most of the advantage is a depth effect". But this average is dominated by the $10N$ value, where $\log M_{\text{loop}}$ is only 0.17 and the ratio is unstable, and the plan keeps the value at the rule's rate as the primary result. The deep model also costs about half the training compute of the looped model: 406M against 835M FLOPs per token at 40M.

About half of the looped model's advantage at equal parameters at 40M thus comes from depth. With each model at its best tested learning rate, depth accounts for all of it at the shortest budget, in the one seed tried: the deep model's lead there, 0.010 nats, is twice the range of this gap across the three seeds at the

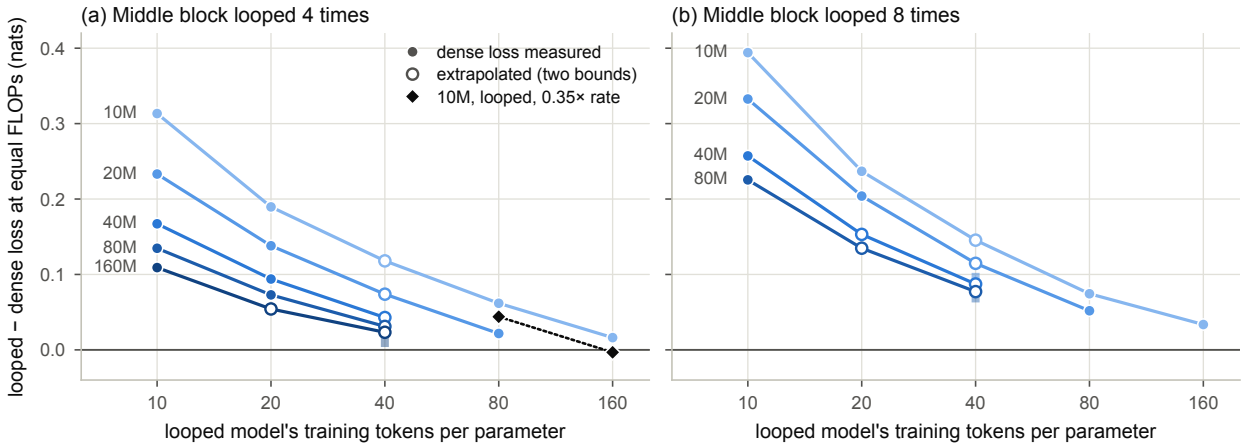


Figure 4: Equal training compute: looped minus dense loss, when the dense model of the same size is trained on the tokens that the looped run's compute would buy (positive: dense better). Filled markers: the dense loss is measured, including the dense models trained longer in S4, to $640N$ (10M) and $320N$ (20M). Open markers with bars: the dense loss is extrapolated, and the bar spans the two extrapolations (from the power law to the extended last segment). Diamonds: the 10M model looped 4 times, trained at 0.35 times the dense rate, the lowest rate S1 tried (mean of two seeds; bars: seed range).

rule's rate (Table 13). At 20M depth explains none of the advantage, and at neither size does depth recover the part that grows with data (Appendix E).

4.4 At equal training compute, the dense model wins all but one measured comparison

At equal parameters, a middle block looped 2, 4 or 8 times costs 1.4–1.5, 2.2–2.4 or 3.7–4.1 times the training FLOPs per token of the dense model. For an equal-compute comparison, we give the dense model the tokens it could have seen with the looped run's compute, and read its loss off the measured dense curve *(exploratory)*.

The grid and its extensions have 320 looped results. For 221 of them, the original dense runs reach the equal-compute point, and at the shared learning rate the dense model won all of them, by at least 0.016 nats. Continuing the 10M dense models to $640N$ and the 20M ones to $320N$ (S4, *pre-specified*) turned 37 more comparisons into measured ones (Figure 4), and the dense model won these as well: all 258 measured comparisons at the shared rate. These comparisons share dense curves, trunks and seeds, so they are not independent.

The dense model's lead shrinks with data: for the 10M 4-loop model trained on $40N$, $80N$ and $160N$, it is +0.119 nats (with the dense loss extrapolated for one seed), +0.062 and +0.016. Table 12 in Appendix D lists the comparisons at the largest budgets.

The remaining comparisons need an extrapolated dense loss, which two extrapolations bound: a power law fitted to each dense curve, which favours looping, and an extension of the curve's last measured segment, which favours the dense model (Appendix D). Of the 99 such comparisons before S4, looping came out ahead in 6 under the power law, by at most 0.021 nats, all at the largest data per parameter. Where S4 measured such comparisons, the measured gaps fall about halfway between the two extrapolations; 62 comparisons remain extrapolated.

With each model at its best tested learning rate (S1), the dense model won 10 of the 11 comparisons (one of them, 40M at $40N$, on an extrapolated dense loss). The eleventh is at 10M and $160N$, with the looped model at 0.35 times the dense rate. That was the lowest rate tried, and the looped model's loss was still falling as the rate went down. There the two are level on average (−0.011 and +0.005 in two seeds). It is the only measured equal-compute comparison in this study that the dense model does not win.

The deep dense models of Section 4.3 cost about half the looped model's compute per token, and they are ahead by 0.11–0.19 nats wherever their measured curves reach the equal-compute point *(exploratory)*.

4.5 At deployment, the loop saves some weight memory but costs decoding time and KV cache for every executed layer

We measured inference on one RTX 4090 in bf16, with KV-cached decoding and a separate cache for every executed layer, so that decoding computes what the models computed in training (S3, *pre-specified*; Appendix F). At these sizes, decoding time follows the number of executed layers. At equal parameters, a middle block looped 2, 4 or 8 times takes 1.5, 2.4–2.6 or 4.5–4.6 times the decoding time per token of the dense model, close to the ratios of executed layers.

At equal quality, the comparison is with the dense model that reaches the same loss when both are trained on 40 tokens per parameter, interpolated between the measured dense sizes. This differs from $N^\star$, which holds the number of training tokens fixed. Against it, the 4-loop model needs less weight memory: 4% less at 20M, rising to 19% at 160M, where the dense model of equal loss is extrapolated. But it needs 2.20–2.41 times the KV cache per token, 2.37–2.47 times the decoding time and about 1.6 times the peak memory at batch 32.

One way to cut this cost is to share the KV cache across loops. Each core layer then keeps one cache for all loops, and every loop reads the earlier tokens' keys and values as they were written at those tokens' last loop. This would remove most of the cache and memory cost, though not the decoding time. A supplementary experiment (S6, *pre-specified*) applied it after training to 20M models of one seed, at the constant-rate checkpoint at $32N$ from which the $40N$ cooldown branches off. The models include a 4-loop model trained at half the dense rate in S1. The comparison with the dense model at the same checkpoint and on the same tokens was added after a review and is *exploratory* (Appendix H). For every model that gains from its loops, sharing costs 2.2 to 7.4 times that gain (0.09–0.17 nats, against gains of 0.019–0.046 over the dense model), so each would end up worse than the dense model of its size. These gains come from one seed, and at 20M the gap between a looped and a dense model varies across seeds by 0.032–0.045 nats (Section 3.3), so the ratios are rough. By the S6 plan's thresholds (an increase above 0.02 nats "degrades", above 0.1 "breaks"), sharing degrades the 8-loop model (by 0.09 nats) and breaks the truncated 8-loop model (0.14), the 4-loop model (0.17) and, at the boundary, the 4-loop model at half the rate (0.10; Appendix F). Sharing costs nothing only for the models trained with a random loop count, whose block contracts to a fixed point (Section 5); these models gain nothing over the dense model. We tested neither of two routes that may avoid this cost: training the loop to converge, and training with a shared cache from the start. Monea et al. (2026) pretrain looped models with a scheme in which only the first loop writes the cache, and find that it costs no quality in their setting.

What the loop buys on text. Where the loop helps at equal parameters, it saves parameters and data but costs compute. From about 40M to 160M, with 20–40 tokens per parameter, each parameter of the 4-loop model is worth about 1.3–1.5 dense parameters, more with more data at most sizes. At 40M, a dense model of the same size and nearly the same depth gets about half of that advantage for half the compute (all of it at the shortest budget once its learning rate is tuned, in one seed), but not the part that grows with data. At equal training compute the dense model wins every measured comparison but one, and at deployment the loop costs decoding time and KV cache for every executed layer. The next two sections test the second claim: that running more loops at inference solves harder problems.

5 On text, extra loops at inference never help

If the loop iterated towards an answer, the state after each loop would carry a better prediction than the state before it, and running more loops at inference would help. At these sizes, it does not. To see what the loop computes instead, we probed trained models loop by loop, on trunk checkpoints at $32N$ before cooldown *(exploratory)*. We read the state after each loop with the model's own output layers (the coda and the output head), with a learned linear map placed before them, and with a learned linear head in their place (Appendix C). Reading it through the model's own output layers, a logit lens, is the same as running

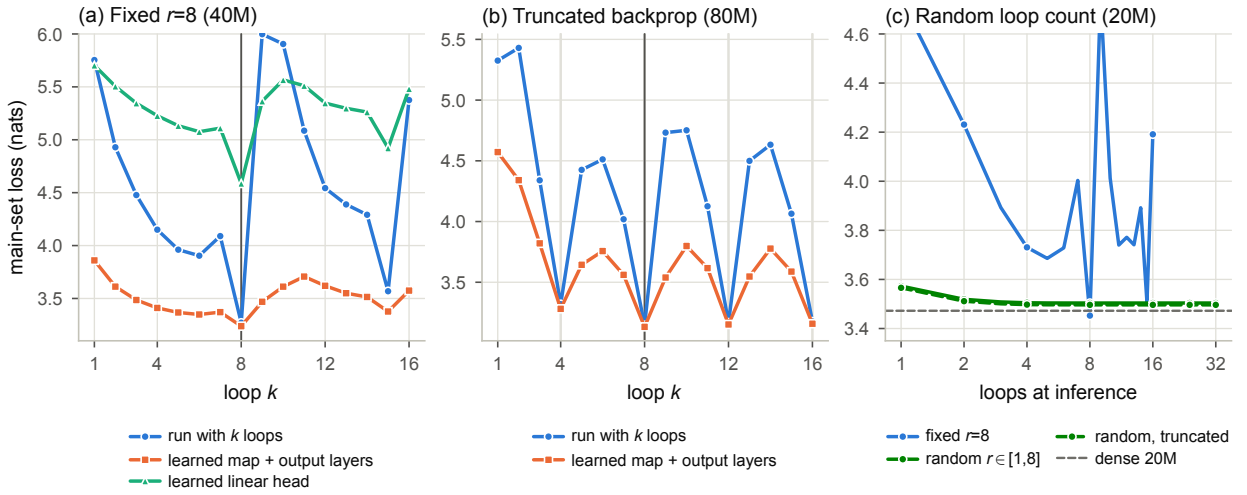


Figure 5: What the looped block computes on text (main-set loss of trunk checkpoints at $32N$; values above the top of a panel are clipped). **(a)** Fixed loop count: a 40M middle block trained with 8 loops (mean of three seeds), read after each loop k by running the model with k loops, by a learned linear map placed before the output layers, and by a learned linear head in their place. The model reaches its low loss only at its training loop count. **(b)** Truncated backpropagation (80M whole stack, $r = 8$, gradients through the last 4 loops): the state returns to almost the same point every four loops. **(c)** Trained with a random loop count (20M middle block, r drawn from 1–8 at every training step), the block contracts to a fixed point: the loss is flat from about four loops on and no better than the dense model, with or without truncation.

the model with that many loops; the learned map is a tuned lens trained on the next token. Figure 5 shows three training regimes.

Trained with a fixed count of 8 loops, the middle block behaves like a fixed-length program. The model reaches its low loss only at 8 loops, and running it with fewer or more loops makes the loss much higher (Figure 5a). Read with a learned map, the state after loop 4 is already within 0.1–0.2 nats of the final loss, and loops 5–7 add little. The last loop is a large non-linear move that rewrites the state into what the output layers expect: for 37–47% of the tokens, the predicted token changes for the last time at loop 8. These patterns held in all three seeds at 20M and 40M, and in the one seed at 80M.

With truncated backpropagation, the whole stack learns a periodic cycle. When it is trained with 8 loops and gradients through only the last four, its state returns to almost the same point every four loops (Figure 5b). The model works at 4, 8, 12 or 16 loops and gains nothing from more loops. The same minima at 4, 8 and 12 loops appear at 20M and in both probed 40M seeds. We did not probe a middle block trained with truncated backpropagation.

Drawing the loop count from 1–8 at every training step, as Huginn does with a different distribution (Geiping et al., 2025), gives a block that contracts to a fixed point. Its loss is flat from about four loops to 32 (Figure 5c; one seed at 20M, with full and with truncated backpropagation). Such a model is robust to the loop count, but at 8 loops, the top of its training range, it is 0.05 nats worse than the model trained with a fixed 8 loops and no better than the dense model. Parcae's models, trained with a sampled loop count at 140M–770M, keep an advantage, so our result, from one seed at one size, may be specific to small models or to our distribution.

Lower learning rates give flatter curves over the inference loop count. In S1, whose plan lists this as an exploratory question (Q5), the loss of the 4-loop model at 10M, 20M and 40M varies by 0.36–0.77 nats over 4–16 loops at 0.5 times the dense rate, and by 3.4–4.6 nats at 1.41 times. At every rate, though, the better of 8 and 16 loops was worse than the training count of 4.

The looped model's gain over the dense model on web text is small and spread across tokens *(exploratory)*. About half of the tokens improve, and the gain is no more concentrated on hard tokens than the difference between two dense seeds is. On math and code text the gains are larger, but there the spread between seeds is 10–100 times larger as well, and the code set is more sensitive to the data order than to the initialisation (S7, *pre-specified*; Appendix C). We draw no conclusions from differences on those sets.

On web text at these sizes, then, the loop gains little from iterating, and running it for more loops than it was trained with never helped.

6 On a serial task, a loop buys extendable depth

To test the second claim where iterating should matter, we built the chain task, a task that needs serial steps, and froze its design, conditions and predictions before its main runs; it uses the same architecture as the language models. Pilot runs then led to three changes before any formal run, all to make the task learnable: the operations became lookup tables instead of modular arithmetic, a curriculum was added, and training was lengthened. The last of these came after a one-seed pilot of all conditions, and the learning-rate protocol was fixed after that (Appendix H).

6.1 The task and the conditions

A problem lists a start value and a sequence of table lookups (Figure 2b), for example `25 ; LUT 7 ; LUT 3 ; LUT 19 ; ... =>` after a start marker. Here `LUT k` means "look up the current value in table T_k", one of 32 fixed random tables that map $\{0, \ldots, 96\}$ to itself. Each lookup is one step: the j-th gives the intermediate value x_j, and a chain of d steps has d lookups and a prompt of $3d + 4$ tokens. The model must output the final value in a single forward pass, and the loss is taken at the answer position only. Two consecutive lookups can form any of the $32 \times 32 = 1024$ compositions of two tables, so a model that skips steps must memorise tables of compositions, which grow rapidly with the number of steps skipped. Because composition is associative, the chain could in principle be computed in fewer serial steps than it has lookups (Liu et al., 2022).

Models are trained on chains of 1–8 steps for 32,000 training steps, with batches of 512 problems sampled online and a curriculum that widens the range of lengths over the first half of training. They are tested on 2,000 newly drawn problems at every length from 1 to 24. Chains longer than 8 steps never occur in training, but short ones repeat: every test problem with 1 or 2 steps, and about half of those with 3, also occurs in training (there are only 3,104 one-step problems).

All models have width 256. The dense models have 8, 20 or 36 layers (6.5M, 16.1M and 29.0M parameters); those with 20 and 36 layers match the executed depth of the looped models with 4 and 8 loops. The looped models have 8 layers (6.6M parameters), with a middle block of four layers looped r times. They are trained with $r = 2$, 4 or 8 loops, with a random loop count drawn from 1–8 at every training step, or with as many loops as the chain has steps ("loops = steps", the regime of Fan et al., 2025). A fourth regime, loops = steps with a few random extra loops (S9), is introduced in Section 6.6. In S5 (*pre-specified*), dense models write the values of the chain out as tokens, the last of them the answer. This chain of thought (Wei et al., 2022) is trained from scratch on the written values, as in scratchpad training (Nye et al., 2021), with 8 layers (the looped models' size) and with 36 (the 8-loop model's executed depth). We also train an 8-layer dense model given d pause tokens before the answer (Goyal et al., 2024).

Every condition has five seeds, and its learning rate was chosen by a sweep on in-range accuracy only. The looped models' rates came out 4–8 times lower than that of the 8-layer dense model, which has the same unique layers (Appendix G). The design fixed five predictions (P1–P5, Section 6.7). A planned control task, pointer chasing, was not learned by any condition tried on it and was dropped (Appendix G).

Table 3: The chain task, trained on 1–8 steps (seed means; chance is 0.01; accuracies rounded, and only the chain-of-thought models are exactly perfect in range). *Largest solvable length*: the longest chain length up to which every length is solved with accuracy ≥ 0.9 (mean, and min–max over five seeds). The frozen design's measure, the largest length solved at ≥ 0.9 without requiring every shorter length, gives the same values in the *Training loops* column and differs only for 2 loops under *More loops* (10.4 instead of 8.6). *Training loops*: at the training loop count (8 for the random loop count, d for loops = steps; dense models as trained). *More loops*: the training loop count or any larger count tried, including d loops, whichever is best for each seed, chosen using the test answers (a reference, not a rule a model could apply); the set of counts was widened after the results (Appendix H). "—": not applicable. *Layer passes*: executed layers times tokens processed for a 12-step problem, that is, a prompt of 40 tokens, plus the 11 written values that precede the answer for the chain-of-thought models (each fed back with a KV cache; the answer itself is not processed) or 12 pause tokens.

			Within the range	Largest solvable length		Beyond the range	
Model	Params	Layer passes	Acc. 1–8 steps	Training loops	More loops	Acc. 12 steps train. / more	Acc. 16 steps train. / more
Equal parameters							
Dense, 8 layers	6.5M	320	0.621	4.2 (4–5)	—	0.06	0.05
Looped, $r = 2$	6.6M	480	0.906	6.4 (6–7)	8.6 (6–11)	0.05 / 0.64	0.04 / 0.33
Looped, $r = 4$	6.6M	800	1.000	8.0 (8–8)	11.8 (10–15)	0.03 / 0.83	0.03 / 0.28
Looped, $r = 8$	6.6M	1440	0.998	9.0 (9–9)	11.2 (11–12)	0.04 / 0.86	0.03 / 0.37
Looped, random loop count	6.6M	1440	0.999	11.4 (10–13)	12.2 (10–15)	0.77 / 0.83	0.08 / 0.49
Looped, loops = steps	6.6M	2080	1.000	12.8 (11–16)	12.8 (11–16)	0.96 / 0.96	0.37 / 0.37
Loops = steps + 0–4 extra (S9)	6.6M	2080	0.999	11.6 (11–12)	12.2 (12–13)	0.90 / 0.94	0.31 / 0.48
Equal executed depth, which here is equal compute							
Dense, 20 layers (as $r = 4$)	16.1M	800	0.910	6.4 (6–7)	—	0.04	0.03
Dense, 36 layers (as $r = 8$)	29.0M	1440	0.999	8.0 (8–8)	—	0.02	0.02
Dense models with written-out reasoning or pause tokens (S5)							
Chain of thought, 8 layers	6.5M	408	1.000	8.0 (8–8)	—	0.01	0.01
Chain of thought, 36 layers	29.0M	1836	1.000	8.0 (8–8)	—	0.02	0.01
d pause tokens, 8 layers	6.5M	416	0.566	4.0 (4–4)	—	0.01	0.01

6.2 Within the training range, a loop adds depth, and a written chain of thought does so more cheaply

At equal parameters, the largest solvable length rises with the number of loops: from 4.2 steps for the 8-layer dense model to 6.4, 8.0 and 9.0 steps with 2, 4 and 8 loops (seed means; Table 3, *pre-specified*). Within the training range, the 8-loop model is at 0.998 where the 8-layer dense model is at 0.62.

At equal executed depth, and so at equal compute, the looped models are at least as good as the dense ones. A dense model of equal executed depth has the same width as the looped model and is trained for the same number of training steps on the same problems. On this task it therefore costs about the same training and inference compute, but it has 2.4–4.4 times the parameters, and if the extra parameters matter, they help the dense model. The 20-layer dense model solves 6.4 steps against 8.0 for 4 loops, though it did not fully learn the training range (0.91). Its learning rate was chosen in a sweep whose runs were half as long as the formal runs, and in that sweep it reached 0.98 (Appendix G). The 36-layer dense model solves 8.0 steps against 9.0 for 8 loops, and the two are level within the range. These lengths are the plan's measure, the solvable length at the training loop count; Section 6.3 discusses the *More loops* columns.

On text, the dense model of equal compute has the looped model's size and more data. Here it has more parameters and the same data, and it does not beat the loop. We do not take these two comparisons as evidence that the loop saves compute, since one dense model did not fully learn the range and the other is level with the loop within it.

A written chain of thought is perfect within the training range and costs less than the loop. It scores 1.000 in every seed and at every learning rate tried, where the 8-layer model that answers directly solves 4 steps. At

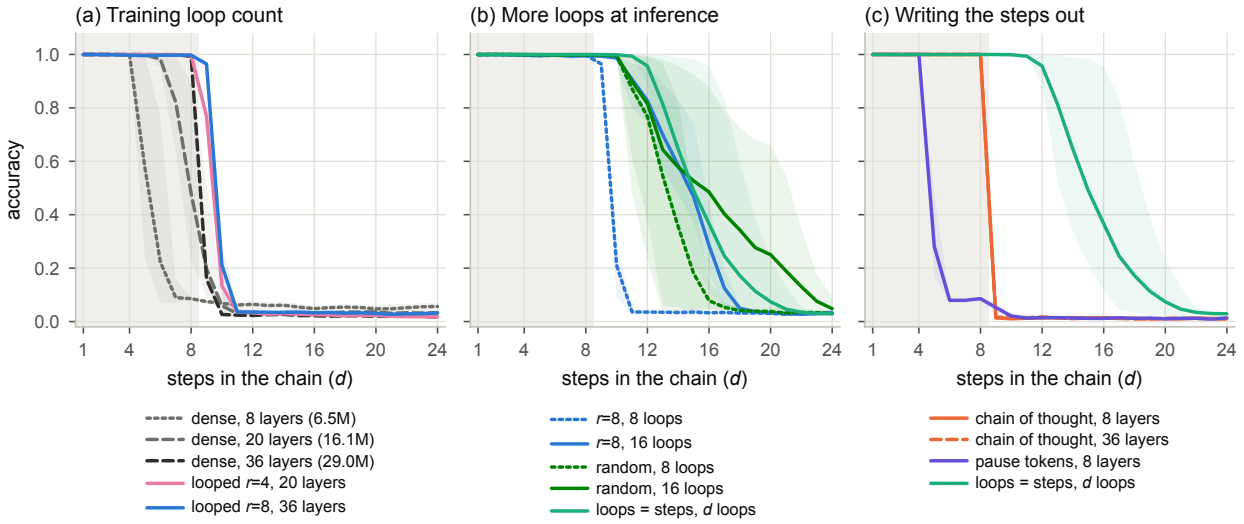


Figure 6: The chain task, trained on 1–8 steps (shaded): accuracy against chain length, mean over five seeds (band: range). **(a)** At the training loop count, looped models solve at least as far as dense models of the same executed depth, and none gets more than one step past the training range. **(b)** With more loops at inference, the looped models solve longer chains; the model trained with as many loops as steps, run with d loops, solves the longest chains at accuracy 0.9. **(c)** Dense models that write the chain out are perfect within the training range and near chance beyond it; pause tokens add nothing (the loops = steps model is replotted for comparison).

any length it needs under a third of the inference compute of the 8-loop model: 408 against 1440 layer passes at 12 steps (Table 3). The comparison favours it, because it is trained with a loss on every intermediate value and the other models on the answer only. The plan stated this difference in advance, and a looped model with intermediate supervision was outside it. Pause tokens add nothing: the pause model solves the same 4 steps as the model without them.

6.3 Beyond the training range, only the loop extends

We say that a model *extends* if, given more computation at inference, it solves chains longer than those it was trained on. At their training loop count, neither the dense models nor the fixed-loop models get more than one step past the training range: at 12 steps, the dense models that answer directly score 0.01–0.06 and the fixed-loop models 0.03–0.05 (Figure 6a).

Run with more loops, the looped models extend (Figure 6b). The model trained with 8 loops solves 9 steps at 8 loops and about 11 with 16 or 32. At 12 steps its accuracy rises from 0.04 to 0.83 at 16 loops, and to 0.86 when the best loop count between 12 and 32 is chosen for each seed using the answers (seed means). The gain is limited, though: at 16 steps the best is 0.37, with a wide spread across seeds. The model trained with a random loop count gains as well (0.08 to 0.49 at 16 steps), against the frozen prediction that it would not. Run with as many loops as the chain has steps, the loops = steps model solves the longest chains at accuracy 0.9 (12.8 steps). Part of this comes from being told how long to run, since its loop count is the number of steps, which the prompt shows. With d loops it is 0.34–0.95 more accurate than with 8 at every length from 10 to 16 (we count from 10 steps, a choice made after the results; at 9 steps the difference is 0.03).

The written chain of thought, perfect within the training range (Section 6.2), does not extend at all. It scores 0.014 at 12 steps with 8 layers and 0.017 with 36, near the chance level of 0.010, on prompts on which the looped models trained with 4 or more loops reach 0.83–0.96 (Figure 6c). S5 had predicted at least 0.7 for the 8-layer model and at least 0.8 for the 36-layer one. The 8-layer model breaks at the start. On problems with more than 8 steps, its first wrong token is one of the first three written ones in 97–98% of the problems

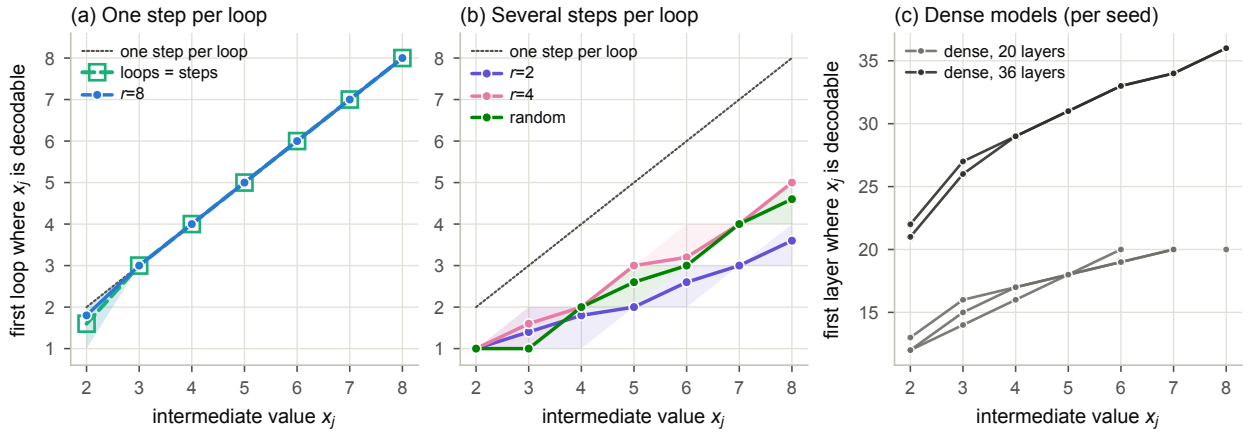


Figure 7: Where the intermediate values appear (probes on 8-step problems; a value counts as decodable at probe accuracy ≥ 0.5). **(a)** With a loop for every step, the chain advances one step per loop. **(b)** With fewer loops than steps, each loop advances several steps (mean over seeds; band: range; looped models probed while running 16 loops). **(c)** Dense models compute the values late in their depth (one line per seed). A seed in which only x_1, not plotted, and the answer are decodable appears as a single point at the last layer; the 20-layer seed with no decodable intermediate value is not shown.

(exploratory), so its errors do not build up along the chain. Because the prompt of this task grows with the number of steps, the chain of thought may be failing to generalise to longer inputs, a known weakness of transformers (Zhou et al., 2024; Cho et al., 2024), rather than to more steps. The comparison cannot say whether a written chain would extend on a task whose input length does not grow; we did not run such a task, and remedies for length generalisation were outside the plan. Pause tokens do not extend either, as S5 predicted.

6.4 The loop computes the chain step by step

For each intermediate value x_j, we trained a linear probe on the state after every loop, on problems with 8 steps and with the looped models running 16 loops (Figure 7). The probes were planned in the frozen design; the summaries below are *exploratory*. Each probe reads three prompt positions around step j (its table number, the separator after it and the next lookup token), and the best of the three counts.

With a loop for every step, the chain advances one step per loop. In the model trained with 8 loops, x_j first becomes decodable (probe accuracy ≥ 0.5) at about loop j, at 1.0–1.1 loops per step in all five seeds. It stays decodable afterwards, with a lowest probe accuracy at later loops of 0.96, averaged over the values. The loops = steps model also computes one step per loop and keeps its values for its d loops (lowest later accuracy 0.92), but it loses them when run longer. It was never trained to keep looping after the last step, which is why it needs exactly d loops.

Models with fewer loops than steps pack several steps into each loop, about 1.6 with 4 loops and 2.4 with 2, in the direction that Zhang et al. (2026) predict. In these models the last values become decodable at the step positions only after the training loop count: at loop 5 with 4 loops, and at loops 3–4 with 2.

The dense models compute the values late, from about layer 21 of 36, and in 5 of the 10 dense seeds (five at 20 layers and five at 36) no intermediate value after x_1 is decodable. These probes are consistent with the extension results: a loop that applies the same step each time can be run for more steps, while a stack that computes the chain in its last layers cannot. A probe shows where a value can be read out; it does not show that the model uses the value.

Table 4: Stopping without the answer: accuracy on 12-step chains (seed means, reporting set). In parentheses: mean loops used, where given; Table 16 gives them for every rule. The best fixed loop count is chosen with the answers; it is a reference, not a rule, and an adaptive rule can exceed it ($r = 2$).

Model	Training loop count	Best fixed count	State change $< 2\%$	State change $< 0.5\%$	Confidence ≥ 0.99
Looped, $r = 2$	0.05	0.62	0.45	0.45 (32)	0.70 (8.3)
Looped, $r = 4$	0.04	0.83	0.65	0.82 (27)	0.81 (9.2)
Looped, $r = 8$	0.04	0.86	0.71	0.84 (28)	0.84 (17)
Random loop count	0.77	0.83	0.73	0.83 (22)	0.83 (14)
Loops = steps (d loops)	0.96	0.96	0.03	0.03 (29)	0.76 (14)[a]
Loops = steps + 0–4 extra (S9)	0.90	0.94	0.52	0.94 (24)	0.93 (14)[b]

[a] 6–7% of the in-range problems lost; [b] 0.998 of the in-range problems kept.

6.5 The learning rate decides whether the loop learns the task within 32,000 training steps

At 10^{-3}, a good rate for the dense models, the 8-loop model of the learning-rate sweep worked only at its training count of 8 loops and at no other count from 2 to 32, like the language models of Section 5. No intermediate value became decodable one loop per step. But it had also learned the training range much worse: 0.54, against 0.999 at 2.5×10^{-4}.

A supplementary experiment (S2, *pre-specified*) asked whether a high rate changes what the looped model learns or how fast it learns. It trained the four looped conditions named in its plan ($r = 4$, $r = 8$, the random loop count and loops = steps) for 32,000 training steps with three seeds. At 10^{-3} they did not learn the training range in 10 of 12 runs; at 5×10^{-4} they learned it in 8 of 12, and those runs mostly extended about as far as the formal runs (Appendix G). A high learning rate mainly keeps the looped model from learning the task within the given training steps, and when the model does learn the task, it mostly learns the same extendable algorithm. By the plan's own criteria, the learning rate changed the solution in one of the eight combinations of condition and rate, the random loop count at 5×10^{-4} (Appendix G).

6.6 Stopping without the answer

Most looped models can decide when to stop from their own confidence, without knowing the answer (Table 4), although we chose this rule after comparing several. Extra loops help in deployment only if their number can be set without the answer. Always running a fixed number of extra loops works for the 8-loop model (Section 6.3) but not for the loops = steps model, which loses its values when run longer (Section 6.4). The alternative is a rule that decides for each problem when to stop.

The frozen design specified one such rule, for the loops = steps model: stop when the state changes by less than a threshold between loops. At the freeze it was implemented as a relative change below 2% at the answer position, with at most 32 loops. A supplementary experiment (S8, *pre-specified*), planned after this rule's results were known and frozen before the comparison, tested it against three other kinds of rule on the 25 checkpoints of the formal runs (five looped conditions, five seeds each). The prediction rule stops when the predicted answer has not changed for m loops, the confidence rule when the model's top probability reaches a threshold, and the vote takes the majority answer over several loop counts. The confidence rule is an early-stopping variant of the maximum-confidence rule of Fan et al. (2025), who mention this variant but do not evaluate it. All rules were fixed in the plan, selected on one problem set and reported on another, drawn independently (Table 4).

The frozen state rule recovers about 80% of the gain of the best fixed loop count at 12 steps for the models trained with 4 or 8 loops, without hurting the training range. It cannot stop the loops = steps model, whose state keeps changing after d loops (accuracy 0.02–0.08 at every length).

The confidence rule does better. At a threshold of 0.99 it recovers 97–98% of that gain for the models trained with 4 or 8 loops or a random count, and on in-range problems it uses fewer loops than their training count.

It brings the loops = steps model to 0.76, but loses 6–7% of the in-range problems. Among the rules that stop, a confidence rule (at 0.99, or at 0.9 for $r = 2$) is the best, or within 0.001 of it, in every condition of S8. A state rule with a lower threshold (0.5%) comes within 0.005 of the confidence rule for 4 and 8 loops and the random loop count, and for $r = 4$ is slightly above it (0.816 against 0.815). But it uses 1.5 to 3 times as many loops. The prediction rule often stops on a wrong answer that no longer changes, and the votes run all 32 loops and never beat the best fixed count.

We singled out the confidence rule only after this comparison. The selection criterion of the S8 plan picked a vote for three of the five conditions, and with votes set aside it picked the confidence rule at 0.99 only for $r = 8$ and the random loop count (Appendix G). Two of the plan's five predictions held: the prediction rule stayed within 1.5 times the training loop count on in-range problems, and the votes did not beat the best fixed count. Contrary to the plan, the prediction rule neither beat the state rule nor recovered 85% of the gain for $r = 4$ and $r = 8$, and the confidence rule took the loops = steps model above the predicted ceiling of 0.2.

The models that extend furthest and the models that stop well came from different training regimes. A supplementary experiment (S9, *pre-specified*) trained the loops = steps model with 0–4 random extra loops after the last step. The resulting model keeps its values when run longer (lowest probe accuracy over 16 loops 0.04 → 0.91) and extends about one step less (11.6 against 12.8). With the confidence rule it reaches 0.93 on 12-step problems (0.89–0.95 in the five seeds) while still solving 0.998 of the problems within the training range. So a few extra loops in training give a model that both extends and stops. The 0.5% state rule is slightly better (0.94 against 0.93, and better in all five seeds), but runs 24 loops against 14. With the frozen state rule, how well the model stops depends on the seed: the rule works in two seeds, fails in two and partly works in the fifth, for a mean of 0.52 at 12 steps. The plan had predicted at least 0.6, and this is the only one of its four predictions that failed. None of the three outcomes the plan anticipated covers this combination of results. Training the model costs about 1.4 times the compute per training step of the loops = steps model.

6.7 The pre-specified predictions

The frozen design made five predictions (Table 15 in Appendix G):

- **P1**: the largest solvable length rises from the dense model through 2, 4 and 8 loops, and reaches at least 16 steps with 8 loops. It holds in direction, but the 8-loop model reached 9 steps.
- **P2**: dense models of equal executed depth are at least as good as the looped models. The looped models solve 1–1.6 more steps, so P2 is not supported.
- **P3**: three predictions about running a model with a loop count other than its training count. (a) Fixed-loop models get much worse. This is half right: they get worse with fewer loops, but the 4- and 8-loop models do not get worse with more. (b) The random-loop model does not gain from more loops on chains longer than 8 steps. This fails, since the model gains (0.08 to 0.49 at 16 steps). (c) The loops = steps model, run with d loops, beats itself run with 8 loops at every length from 9 to at least 16. By a criterion chosen after the results, this holds from 10 steps on (Section 6.3).
- **P4**: the probe table of the loops = steps model is diagonal (x_j appearing at loop j; Figure 7a), and that of the fixed-loop models is not. If a value counts as appearing at the loop where it first becomes decodable, a reading chosen after the results, P4 is half right, because the 8-loop model's table is diagonal too. Under the design's own wording, which takes the loop where a value is decoded most accurately, P4 would hold.
- **P5**: the differences between looped and dense models of equal parameters are far larger than on text. This holds.

On the chain task, then, the loop adds serial depth, and probes find the chain's intermediate values in its state one after another, which is consistent with step-by-step computation. Within the training range (Section 6.2), this depth saves no compute over a dense model of the same executed depth and costs more than a written chain of thought. Beyond the range (Section 6.3), the loop is the only form of extra computation we tried that extends. Dense models that answer directly cannot run longer, even with the 8-loop model's

executed depth, and the chain of thought fails from its first written tokens, possibly because its prompts grow longer than any seen in training.

7 Discussion: when extra loops at inference help

The same architecture is parameter-efficient on text and, on the chain task, carries out a serial computation that more loops can extend. Our interpretation of the difference, which we have not tested, is that extra loops at inference help when the model's errors come from too few serial steps and the loop can run longer without destroying its state, and that using them in deployment also needs a signal for when to stop.

The first condition is the hardest to measure on its own, because here it is inferred from where extra loops help. On the chain task, the dense models know every table, and within the training range their errors come from too few serial steps: one more loop is one more step. On web text at these sizes we found no sign of such errors. The per-token gain is spread across tokens, and extra loops never helped, at any learning rate. The larger gains on math and code text hint at more errors of this kind there, but at this size the results on those sets vary mainly with the initialisation and the data order.

Whether a loop can run longer depends on the kind of iteration that training gives it. On text, a middle block with a fixed loop count becomes a fixed-length program, a whole stack trained with truncated back-propagation a periodic cycle, and a middle block with a random loop count a fixed point. The fixed point can be run for any number of loops but gains nothing on text from the extra ones, the cycle works only at multiples of its period, and the fixed-length program works only at its training count. On the chain task, models trained with as many loops as steps lose their values when run longer, and a few random extra loops in training fix that.

To spend extra loops only where a problem needs them, a deployed model must decide how many to run. A fixed number of extra loops works for the 8-loop model but not for the loops = steps model, which loses its values when run longer. On the chain task, the model's own confidence served as such a signal for most training regimes *(exploratory)*, including the one that extends furthest once it is trained with a few loops past the last step. We did not test stopping rules on text, where extra loops did not help.

The chain task meets these conditions once the model is trained for them. Web text at these sizes showed no sign of the first, and the one text regime that can run longer, the random loop count, gained nothing from it. Two follow-ups that we did not run would test this interpretation. One is a serial task whose input length does not grow with the number of steps, or this task with prompts padded to a fixed length. It would also show whether a written chain of thought extends when the prompt does not get longer, which is the control that Section 6.3 lacks and that no frozen plan included. The other is training on text that needs more serial steps, such as mathematics and code, compared at equal compute.

On text, where extra loops do not help, a loop still has value at equal parameters, and how much depends on size, data and the dense model it is compared with (Section 4). Some effective-parameter laws let the factor applied to the looped parameters depend on the loop count but not on model size or training tokens. Such laws fit the multiplier of the larger models but not its rise with data, and they do not fit the smaller models at all; the single pre-registered φ is an average over these two regimes. Studies at one scale, or against dense references of one depth, will underestimate or overestimate the loop, depending on their scale and reference.

Practical guidance. Within the range we measured:

- On text, use a loop when parameters are the binding constraint, rather than data, compute or latency. Its advantage at equal parameters grows with model size and tokens per parameter, but a dense model was better at equal training compute in every measured comparison but one (Section 4.4), and at deployment each executed layer costs decoding time and KV cache.
- Try a deeper dense model of the same size first. At 40M it gave about half of the advantage for half the training compute, and at the shortest budget (10 tokens per parameter), with its learning rate tuned, all of it (one seed). At 20M, at width 320, it was worse than the 8-layer model.

- If you loop, loop a middle block with full backpropagation. At similar executed depth, the middle block won every comparison with the whole stack (Table 5). Start with four loops on text; at 40M–80M, eight did about as well at about 1.75 times the training compute. Judge at the target scale, since small-scale ablations underestimate the loop.
- Tune the looped model's learning rate separately, especially at small scale and on tasks that need serial steps. At a rate that suited the dense model, most looped runs did not learn the chain task within the given training steps.
- Do not expect extra loops at inference to help on text after standard training. Sweep the loop count at inference to see whether a model can use more loops, and probe it to see whether it computes step by step even if it cannot.
- If a model must decide how long to run, its confidence was the most useful stopping signal here *(exploratory)*. Training a few loops past the last step made it work for the loops = steps model in all five seeds. Within the training range of our task, a written chain of thought, supervised on every intermediate value, was the cheaper choice; its failure beyond the range probably came from the longer prompts and says little about inputs of fixed length.

8 Limitations, pre-registration and reproducibility

- **Scale, data and evaluation.** The models have 10M–160M parameters and a 16k vocabulary, far below the sizes at which looped language models are deployed. They are trained on one source (FineWeb-Edu, no document seen twice) and evaluated by validation loss only, without downstream benchmarks. The trends with size may bend. There are five sizes, one seed at 80M and one looped cell at 160M; the pre-registration's option of three seeds at 80M for the cells that decide H1 and H2, if time allowed, was not used. The random-loop language models and the KV-sharing test have single seeds. Data-limited or repeated-data regimes, where loops may matter more (Chen et al., 2026b; Gülbahar et al., 2026), are not covered. Seeds change only the initialisation, except in the data-order experiment.
- **Probes and measures on text.** The mechanism on text was probed on trunk checkpoints at $32N$, before cooldown, at 20M–80M; cooled-down models and 160M were not probed, and S1's loop-count sweep also uses trunk checkpoints. The 160M multiplier and the 160M deployment ratios rest on a dense size extrapolated beyond the largest dense model. M counts the output head, which lies outside the looped block (34% of N at 10M).
- **Depth on text.** The dense ruler has eight layers at every size. The deeper dense models of S10 cover two sizes, with one shape each and three seeds: 18 layers against the looped model's 20 executed layers, and narrow at 20M. Their learning rate was checked with one seed. At 40M the best rate was the highest one tried, the plan had no rule to extend the range, and no further rate was run. At 80M–160M, where the multiplier is largest, the share due to depth is unmeasured. No dense control of equal executed depth at the looped model's width, which would have more parameters, was trained on text.
- **Equal compute.** 62 of the 320 equal-compute comparisons rest on an extrapolated dense loss, bounded by two extrapolations, and the measured comparisons are not independent of each other.
- **Learning rates.** The dense rates come from a sweep at one budget. The learning-rate check of the ten looped cells at 20M (Appendix B) tried rates only up to the dense rate, which nine of the ten preferred. In S1, the rate of one cell, the 4-loop model, was re-tuned at 10M–40M, with one seed (two at 20M). There the shared rate understated the looped model by up to 0.024 nats at 10M, and at 10M, at $80N$ and $160N$, the looped model's best rate was the lowest tried. The other cells and 80M–160M use the dense rates. On the synthetic task, ties in the learning-rate criterion went to the lowest rate, several models were best at the lowest rate tried, and how far a model extends is sensitive to the rate.
- **One synthetic task family**: chains of table lookups, with 6.6M-parameter looped models. How far a model extends varies widely across seeds, and a planned pointer-chasing control was not learned and was dropped. Probes show where a value can be read out, not that the model uses it. The chain-of-thought comparison covers one task and one recipe, and its compute comparison counts layer passes, not latency. Its prompt grows with the number of steps, so whether a written chain extends on a task whose input length does not grow was not tested. The chain-of-thought models were supervised on every intermediate

value and got no help with length generalisation, and continuous thoughts (Hao et al., 2025) were not tried.

- **Deployment** was measured on one GPU type, for models of at most 160M parameters, with one KV cache per executed layer. KV sharing was tested only as a change after training, at 20M.
- **Exploratory analyses.** The multiplier, the equal-compute bounds and the probes of the language models were chosen after seeing data. The stopping comparison (S8) was planned after the frozen rule's results were known, though frozen before it ran. Its selection criterion picked the confidence rule at 0.99, which the text highlights, only for $r = 8$ and the random loop count, and only once votes are set aside. How P3(c) and P4 are judged, and the set of loop counts behind the *More loops* columns of Table 3, were decided after the results. The S10 values at each model's best learning rate were computed by a script written after the results, from a rule its plan fixed. The verdicts on the four pre-registered hypotheses do not depend on any of these exploratory analyses.

Pre-registration, deviations and reproducibility. The pre-registration was frozen on 2026-09-23, before any training run, and its SHA-256 was committed to the repository the same day. The synthetic design was frozen on 2026-10-09 and the three supplementary plans on 2026-10-13, 2026-10-15 and 2026-10-27, each before its runs. Every later change is dated in an append-only deviation log, with its reason and its effect on the conclusions (Appendix H). The log includes the changes made before any data were collected, and the errors, overstatements and incomplete disclosures found after the runs and corrected; Appendix H lists those that changed a verdict or a claim. Code, configurations, the per-run results and training logs of all 242 language-model runs and of the 135 synthetic-task runs of the learning-rate sweeps, the formal runs and the supplementary experiments, the fits, and the scripts that make the fits and the figures are available in the repository at `https://github.com/bethehand/looped-scaling`; the remaining tables and numbers are computed from these results. Fits and figures run on a laptop CPU; the model checkpoints (about 560 GB) are not included.

Use of AI assistance. An AI assistant (Claude, by Anthropic) drafted the code and text for this study. The author chose the research question and its scope, decided every design choice recorded in the pre-registered documents and approved each of them before it was frozen, decided which experiments to run and which not to, provided the hardware, checked the paper against the results and takes full responsibility for the content.

References

Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, J. Zico Kolter, and Roger Grosse. Path Independent Equilibrium Models Can Better Exploit Test-Time Computation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2211.09961.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*, 2025a. arXiv:2410.20672.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*, 2025b.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep Equilibrium Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1909.01377.

Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to Ponder. In *8th ICML Workshop on Automated Machine Learning*, 2021. arXiv:2107.05407.

Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end Algorithm Synthesis with Recurrent Networks: Logical Extrapolation Without Overthinking. *arXiv preprint arXiv:2202.05826*, 2022.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv preprint arXiv:2303.08112*, 2023.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. *arXiv preprint arXiv:2502.02737*, 2025.

Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla Scaling: A replication attempt. *arXiv preprint arXiv:2404.10102*, 2024.

Yanbei Chen, Anirudh Goyal, and Raghuraman Krishnamoorthi. Scaling Laws for Looped Mixture of Experts. *arXiv preprint arXiv:2609.40316*, 2026a.

Zixi Chen, Akshay Vegesna, Samip Dahal, and Andrew Gordon Wilson. How Model Growth, Recursion, and Boundary Operators Influence Scaling Exponents. *arXiv preprint arXiv:2609.19107*, 2026b.

Hanseul Cho, Jaeyoung Cha, Pranjal Awasthi, Srinadh Bhojanapalli, Anupam Gupta, and Chulhee Yun. Position Coupling: Improving Length Generalization of Arithmetic Transformers Using Task Structure. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.20671.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1807.03819.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped Transformers for Length Generalization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.15647.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*, 2025.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training Language Models With Pause Tokens. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02226.

Alex Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*, 2016.

Serdar Gülbahar, Lukas Edman, and Alexander Fraser. Squeezing More from Limited Data with Recursive Transformers. *arXiv preprint arXiv:2608.26973*, 2026.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro von Werra, and Martin Jaggi. Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.18392.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2412.06769.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*, 2022.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*, 2026.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.

Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers. In *Conference on Language Modeling (COLM)*, 2026. arXiv:2604.07822.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *arXiv preprint arXiv:1909.11942*, 2019.

Ryan Lee, Jacob Biloki, Edward J. Hu, and Jonathan May. Sparse Layers are Critical to Scaling Looped Language Models. *arXiv preprint arXiv:2605.09165*, 2026.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers Learn Shortcuts to Automata. *arXiv preprint arXiv:2210.10749*, 2022.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14905.

Joe Logan. Per-Token Fixed-Point Convergence in Depth-Recurrent Transformers. *arXiv preprint arXiv:2607.14427*, 2026.

Andrei Marchenko, Viacheslav Bezrukov, Oleg Kashurin, Inessa Fedorova, Dmitry Bocharov, Yuliana Shakhvalieva, Maria Tikhonova, and Valerii Ternovskii. Closing the Loop: Practical Training Recipes for Looped Language Models. *arXiv preprint arXiv:2610.00673*, 2026.

Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers Can Do Arithmetic with the Right Embeddings. *arXiv preprint arXiv:2405.17399*, 2024.

William Merrill and Ashish Sabharwal. The Expressive Power of Transformers with Chain of Thought. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07923.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*, 2026.

nostalgebraist. interpreting GPT: the logit lens. LessWrong blog post, 2020. URL `https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens`.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show Your Work: Scratchpads for Intermediate Computation with Language Models. *arXiv preprint arXiv:2112.00114*, 2021.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv preprint arXiv:2406.17557*, 2024.

Jacob Pfau, William Merrill, and Samuel R. Bowman. Let's Think Dot by Dot: Hidden Computation in Transformer Language Models. *arXiv preprint arXiv:2404.15758*, 2024.

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving Discrepancies in Compute-Optimal Scaling of Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.19146.

Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling Laws For Stable Looped Language Models. *arXiv preprint arXiv:2604.12946*, 2026.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.17416.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can You Learn an Algorithm? Generalizing from Easy to Hard Problems with Recurrent Networks. *arXiv preprint arXiv:2106.04537*, 2021.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. *arXiv preprint arXiv:2604.21106*, 2026.

Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R. Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. Hugging Face dataset card, 2023. URL `https://huggingface.co/datasets/cerebras/SlimPajama-627B`.

Shaowen Wang, Bingrui Li, Tansheng Zhu, Ge Zhang, Wenhao Huang, Shen Yan, and Jian Li. Loop-μP: Residual Scaling and Hyperparameter Transfer in Looped Transformers. *arXiv preprint arXiv:2606.18524*, 2026a.

Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers. *arXiv preprint arXiv:2609.01343*, 2026b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*, 2022.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*, 2025.

Qifan Yu, Zhenyu He, Sijie Li, Xun Zhou, Jun Zhang, Jingjing Xu, and Di He. Enhancing Auto-regressive Chain-of-Thought through Loop-Aligned Reasoning. *arXiv preprint arXiv:2502.08482*, 2025.

Tong Zhang, Junhao Hu, Yun Peng, and Tao Xie. When Does Recurrence Become an Algorithm? Convergence Selection in Weight-Tied Looped Transformers. *arXiv preprint arXiv:2607.20594*, 2026.

Yongchao Zhou, Uri Alon, Xinyun Chen, Xuezhi Wang, Rishabh Agarwal, and Denny Zhou. Transformers Can Achieve Length Generalization But Not Robustly. *arXiv preprint arXiv:2402.09371*, 2024.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*, 2025.

A The pre-registered hypotheses and decision rules

The pre-registration (frozen 2026-09-23; an English translation of the binding original is in the repository, with the commands that recover the original from the dated commit history and verify its hash) fixed the grid of Section 3.2, the fitting protocol of Section 3.3 and these decision rules. Before any data, four changes were made and dated in the deviation log: the training data moved from sample-100BT to sample-10BT, which removed the $80N$ point of the 160M dense model; the data point moved from the mean of a cooldown's last three evaluations to its final evaluation, the three-point mean becoming a robustness variant (Table 10); a variant that counts N without the output head was added; and the exploratory code validation set changed from the-stack, which is gated, to github-code-clean. The pre-registration's five pre-run checks all passed (the 40M half of its second check, that a middle block looped 4 times beats the dense model of its size at $10N$, on 2026-10-07 with main-grid results, under a rule of 2026-09-26).

Table 5: The four pre-registered hypotheses, their frozen decision rules and their verdicts.

Hypothesis	Decision rule for support	Verdict
H1: truncated backpropagation lowers φ	$\varphi_{\text{trunc}} < \varphi_{\text{full}}$ with non-overlapping 95% bootstrap intervals, under both equal-token and equal-FLOP accounting; if only under equal tokens, the effect is attributed to the reallocation of FLOPs.	Not supported. Middle block 0.00 vs 0.13 (predicted direction, intervals overlap); whole stack 0.17 vs 0.00 (opposite direction). The equal-FLOP accounting agrees.
H2: a looped middle block has a higher φ than a looped whole stack	$\varphi_{\text{mid}} > \varphi_{\text{whole}}$ with non-overlapping intervals; raw-loss pairs of similar executed depth (middle block $r = 8$ vs whole stack $r = 4$; middle block $r = 4$ vs whole stack $r = 2$) are reported alongside, and a disagreement is reported as "depends on the measure".	Depends on the measure. φ is mixed; raw losses at similar executed depth (36 against 32 and 20 against 16 layers) favour the middle block in 24 of 24 comparisons, by 0.03–0.19 nats, less at larger sizes.
H3: the gain from extra loops saturates in r	The geometric form has the lower extrapolation error in both directions (10M–40M predicting 80M; 20M–80M predicting 10M) and an information-criterion difference above 10.	Not supported. The geometric form predicts 80M from 10M–40M better (0.0085 vs 0.0096 RMSE) and 10M from 20M–80M slightly worse (0.0203 vs 0.0200); the data do not separate the two forms.
H4: the compute-optimal r	Under matched training FLOPs the optimal $r \leq 2$, under matched deployment FLOPs larger; computed from the fitted law, without a test.	First half holds trivially (the dense model, $r = 1$, is optimal under training compute); second half not supported (it is optimal under deployment compute too).

A factor effect counts only if it exceeds both the bootstrap half-width and the spread of φ fitted separately to the three seeds at 20M and 40M. The pre-registration's rule for the 10M rung, to report fits with and without it, the latter as primary, if its residual in the ruler fit exceeds twice that of the other rungs, does not apply: the rung's root-mean-square residual in log loss is 0.011 against 0.008 for the other rungs pooled and 0.012 for the largest of them (160M); this check was first made on 2026-10-28 (`fit/ruler_residuals.py`; Appendix H). The information criterion of H3 turned out to be computable only as a proxy from the Huber loss, which is not a likelihood; it is not used as evidence (Appendix H). Two rules were added during the grid, after the first spikes (2026-09-26) and after the collapse (2026-09-30), each before the results it governs existed, and are dated in the deviation log: spikes are reported per run with a robustness fit without the runs that have them, and a cooldown result enters no fit if its branch starts after a collapse or ends more than 1.0 nats above the lowest trunk loss before it (four results of two runs: three of a 40M whole stack looped 8 times with truncation, seed 43, which collapsed, and the $10N$ result of a 40M whole stack looped 8 times with full backpropagation, seed 44, after a severe spike). Only two of the four are equal-token results (the $40N$ result of seed 43 and the $10N$ result of seed 44), so the equal-token fits use 352 results; the other two are equal-FLOP results.

B Training details

Table 6 lists the sizes and Table 7 the supplementary experiments.

Table 6: Model sizes and training settings on text. Size names give the approximate non-embedding parameters; N also counts the output head (Porian et al., 2024), which is large at small widths with a 16k vocabulary. The looped models of a size have the same unique layers plus a $2w^2$ injection map (under 2% of N). The learning rate is that of the embedding and output layers, frozen after a 33-run sweep; the hidden layers use 1.22 times it. Warm-up is $2N$ tokens.

Size	Width w	Heads	N	Tokens per step	Learning rate	Seeds (dense / looped)
10M	320	5	15.4M	81,920	2.19×10^{-3}	3 / 3
20M	448	7	26.8M	131,072	1.53×10^{-3}	3 / 3
40M	640	10	50.1M	204,800	1.83×10^{-3}	3 / 3
80M	896	14	92.7M	327,680	2.03×10^{-3}	1 / 1
160M	1280	20	179.6M	524,288	1.56×10^{-3}	2 / 2 (middle block, $r = 4$)

Table 7: The ten supplementary experiments, each run under a plan frozen before its runs (the S8 plan was written after the frozen stopping rule's results on the same checkpoints were known).

	Plan frozen	What it adds	Where
S1	2026-10-13	Learning rates re-tuned for the dense model and a middle block looped 4 times	§4.2, App. D
S2	2026-10-13	Learning rate and learning of the chain task, three seeds	§6.5
S3	2026-10-13	Inference cost	§4.5, App. F
S4	2026-10-15	Dense models continued to $640N$ (10M) and $320N$ (20M)	§4.4, App. D
S5	2026-10-15	Chain of thought and pause tokens on the chain task	§6.2, §6.3
S6	2026-10-15	KV cache shared across loops after training	§4.5, App. F
S7	2026-10-15	Data order	§5, App. C
S8	2026-10-15	Answer-free stopping rules	§6.6, App. G
S9	2026-10-15	Loops = steps with 0–4 random extra loops	§6.6
S10	2026-10-27	Dense models with 18 layers and the looped model's parameters	§4.3, App. E

Data. The three further validation sets of 20M tokens each come from a 6B-token sample of SlimPajama, from FineMath (finemath-4plus) and from GitHub code (github-code-clean, `https://huggingface.co/datasets/codeparrot/github-code-clean`).

Extensions. The extensions continue the 10M middle blocks looped 4 and 8 times (two seeds) to $80N$ and $160N$, a 10M whole stack looped 4 times and a truncated one looped 8 times (one seed) to the same budgets, the 10M dense model to $160N$ and the 20M middle blocks looped 4 and 8 times (two seeds) to $80N$.

Learning rates. A sweep of 33 runs at $10N$ (30 dense runs at five rates per width, a $\beta_2 = 0.99$ group at 20M, and three looped runs at 80M) fixed the rate per width (Table 6); the minima of all five widths lie inside the swept range, and $\beta_2 = 0.95$ beat 0.99 by 0.008 nats. The hidden layers use $\eta_0(8/12)^{-1/2} \approx 1.22\,\eta_0$. A check for looped cells, whose decision rule was written before its runs on 2026-09-25, two days after the pre-registration (20M, ten cells at 0.5, 0.71 and 1 times the dense rate, $10N$), found no better rate by the rule fixed in advance (pooled best multiple 0.97, threshold 0.84), so all looped language models use the dense rates. Nine of the ten cells were best at $1\times$, the top of that range, so higher rates were not excluded. The residual scaling of Section 3.1 is meant to keep the best rate independent of r: we found no downward shift of the best rate in this check, but at 10M with longer training the looped model preferred lower rates (Section 4.2).

FLOP accounting. Training FLOPs count two FLOPs per multiply-add over every executed layer, the attention terms and the output head, plus a backward pass of twice the forward cost for every layer that receives gradients; with truncated backpropagation the backward pass covers the prelude, the coda, the output head and the last $r/2$ loops.

The grid at equal parameters. Table 8 gives the looped minus dense loss per cell and size, averaged over the three budgets.

Table 8: Looped minus dense loss at equal parameters and tokens, mean over the budgets $10N$, $20N$ and $40N$ and over seeds (negative: looped better). *Includes a seed with a severe loss spike; without it, -0.029.

Cell	10M	20M	40M	80M
Middle block, $r = 2$	$+0.068$	$+0.025$	-0.010	-0.015
Middle block, $r = 4$	$+0.055$	$+0.002$	-0.030	-0.039
Middle block, $r = 4$, truncated	$+0.079$	$+0.031$	$+0.006$	$+0.009$
Middle block, $r = 8$	$+0.031$	-0.003	-0.033	-0.033
Middle block, $r = 8$, truncated	$+0.039$	$+0.001$	-0.022	-0.018
Whole stack, $r = 2$	$+0.172$	$+0.079$	$+0.021$	$+0.003$
Whole stack, $r = 4$	$+0.150$	$+0.085$	$+0.047$	$+0.025$
Whole stack, $r = 4$, truncated	$+0.044$	$+0.003$	-0.014	$+0.017$
Whole stack, $r = 8$	-0.074	-0.055	$+0.214^{*}$	-0.003
Whole stack, $r = 8$, truncated	-0.053	-0.041	-0.042	-0.020

Two observations from the grid. Truncated backpropagation, predicted to hurt, helped whole-stack loops with four loops at every size (Table 8): at 20M a whole stack looped 4 times reached 3.646 with truncation and 3.783 with full backpropagation at $10N$ (three-seed means); with eight loops it was slightly worse at 10M–20M. At 10M the only looped cells better than the dense model on average over the three budgets were whole-stack loops with $r = 8$. Truncation and whole-stack loops also brought instability: 30 of the 111 runs had at least one loss spike, 25 with truncated backpropagation (16 of these whole-stack loops) and five whole-stack loops with $r = 8$ and full backpropagation; one run collapsed. In an exploratory follow-up, a 10M whole stack looped 4 times with truncation, trained at half the learning rate in three seeds, had no spike at all (4, 1 and 10 at the original rate); it was worse by 0.044 nats at $10N$ and better by 0.064 at $40N$, which cannot be credited to the missing spikes alone, since longer budgets also favour a lower constant rate. Table 9 lists the runs with spikes.

Table 9: The 30 grid runs with at least one loss spike (the rule of 2026-09-26, Appendix A): number of spikes and the largest rise of the training loss (the mean over 10 steps, as logged) between consecutive log lines, in nats. The other 81 grid runs had none; the full per-run table, including the extension and supplementary runs, is `results/spikes.csv` in the repository.

Size	Cell	Seed	Spikes	Rise	Size	Cell	Seed	Spikes	Rise
10M	Middle block, $r = 4$, trunc.	42	1	1.17	40M	Middle block, $r = 4$, trunc.	42	1	3.02
10M	Middle block, $r = 4$, trunc.	43	11	2.35	40M	Middle block, $r = 4$, trunc.	43	3	2.84
10M	Middle block, $r = 4$, trunc.	44	5	2.25	40M	Middle block, $r = 4$, trunc.	44	5	4.30
10M	Whole stack, $r = 4$, trunc.	42	4	1.97	40M	Whole stack, $r = 4$, trunc.	42	5	2.87
10M	Whole stack, $r = 4$, trunc.	43	1	2.64	40M	Whole stack, $r = 4$, trunc.	43	2	2.20
10M	Whole stack, $r = 4$, trunc.	44	10	3.20	40M	Whole stack, $r = 8$, full	42	1	2.79
10M	Whole stack, $r = 8$, trunc.	44	1	1.10	40M	Whole stack, $r = 8$, full	43	1	5.61
20M	Middle block, $r = 4$, trunc.	43	2	1.92	40M	Whole stack, $r = 8$, full	44	1	2.80
20M	Middle block, $r = 4$, trunc.	44	2	1.31	40M	Whole stack, $r = 8$, trunc.	42	4	3.93
20M	Whole stack, $r = 4$, trunc.	42	1	2.06	40M	Whole stack, $r = 8$, trunc.	43	7	4.84
20M	Whole stack, $r = 4$, trunc.	43	1	1.46	40M	Whole stack, $r = 8$, trunc.	44	6	2.50
20M	Whole stack, $r = 8$, full	43	2	1.79	80M	Middle block, $r = 4$, trunc.	42	4	3.45
20M	Whole stack, $r = 8$, trunc.	42	1	1.67	80M	Whole stack, $r = 4$, trunc.	42	3	4.61
20M	Whole stack, $r = 8$, trunc.	43	3	3.97	80M	Whole stack, $r = 8$, full	42	2	4.50
20M	Whole stack, $r = 8$, trunc.	44	1	1.69	80M	Whole stack, $r = 8$, trunc.	42	13	2.18

C Fits and robustness

The fits use a Huber loss in log space ($\delta = 10^{-3}$), 500 random starts and 1000 block-bootstrap resamples. The primary ruler, fitted to 54 dense results, is $E = 0.050$ (at the lower bound of the fit), $A = 6.98$, $\alpha = 0.054$, $B = 1065$, $\beta = 0.370$; the joint fit over all 352 results gives $E = 1.94$, $A = 61.7$, $\alpha = 0.267$, $B = 197$, $\beta = 0.266$. Under the constant-φ model, setting every φ to 0 raises the Huber loss only from 0.00352 to

0.00379, and setting it to 1 raises it to 0.0142. Table 10 gives the primary φ under the robustness variants of the pre-registration and of the deviation log, each fixed before the results it governs. Fitted separately to each seed, φ varies by 0.10–0.21 per loop type at 20M and 0.00–0.11 at 40M; the factor effects of Table 2 (0.13–0.17) do not exceed the 20M spread. Under the convention that counts N without the output head (0.07 / 0.00 / 0.00 / 0.14), H1 and H2 read the same way, so the verdicts do not depend on the parameter convention.

Table 10: φ (primary fit) under robustness variants, in the order middle block full / middle block truncated / whole stack full / whole stack truncated.

Variant	φ
Primary analysis	0.13 / 0.00 / 0.00 / 0.17
Without the 30 runs with loss spikes	0.13 / 0.11 / 0.00 / 0.22
Mean of the last three evaluations (the pre-registered data point)	0.14 / 0.00 / 0.00 / 0.17
Second validation set (SlimPajama)	0.31 / 0.18 / 0.02 / 0.30
Without $r = 8$	0.07 / 0.00 / 0.00 / 0.07
Half the budget	0.08 / 0.00 / 0.00 / 0.15
N without the output head	0.07 / 0.00 / 0.00 / 0.14
Equal-FLOP accounting	0.13 / 0.08 / 0.00 / 0.16

For H3, the pre-registered extrapolation (10M–40M to 80M and 20M–80M to 10M, with 160M, where the pre-registered grid has only a dense model, in neither training set) gives a root-mean-square error of 0.0096 and 0.0200 for r^{φ} and 0.0085 and 0.0203 for the geometric form. Five of the seven robustness variants show the same split; without $r = 8$ the split is reversed, and on the second validation set r^{φ} is better in both directions. For H4, with the deployment budget read at training tokens equal to 20 times the executed parameter count and in the limit of infinite data (assumptions fixed on 2026-10-07), the fitted law makes the dense model optimal at every training budget from 10^{17} to 10^{20} FLOPs, with the smallest available r the best looped choice, 0.00–0.17 nats behind; under deployment compute the dense model is optimal as well, with either set of ruler parameters.

Loop-by-loop probes of the language models. The state after each loop k is read in three ways: with the model's own output layers (the logit lens of nostalgebraist, 2020, which here is the same as running the model with k loops), with a learned linear $w \times w$ map in front of the output layers (in the spirit of the tuned lens of Belrose et al., 2023, but trained on the next token rather than distilled from the model's own output), and with a learned linear head; the maps are trained on 10M unseen training tokens and evaluated on 2M tokens of the main validation set.

The local exponent. Per looped parameter, the local exponent φ_{loc} of the middle block looped 4 times, counting N with the output head, is negative where $M < 1$ (-0.84 at 20M with 10 tokens per parameter) and 0.22–0.55 from 40M on.

The multiplier plane. A least-squares plane in $\log_2 M$ over the middle block looped 4 times gives a factor of 1.21 per doubling of model size and 1.14 per doubling of tokens per parameter (1.16 with 8 loops), with classical 95% intervals, which include the plane's misfit, of 1.13–1.30 and 1.05–1.24; by these intervals the product term is indistinguishable from zero. On the plane the loop breaks even ($M = 1$) at about $N = 35$M with 20 tokens per parameter and at about $N = 12$M, below the smallest model, with 80; the measured points agree at 80 (1.04 for the 10M model, 1.05 on the plane) but reach $M = 1$ earlier at 20 (the 20M model is at 0.99, the plane at 0.93). At 10M with 10 tokens per parameter the two dense models measured at that number of tokens are out of order (the 20M one, at 5.7 tokens per parameter, is worse than the 10M one), so $N^{\star}$ cannot be extrapolated either.

Published multipliers. Published equal-parameter results mostly restate to a similar level in comparable settings, but their trends differ: Parcae's values fall with size at about 80 tokens per parameter, Ouro's rise with size while its tokens per parameter fall, and small blocks trained on far more tokens per parameter reach

higher values. Where a paper reports a non-looped ladder (its dense ruler), we interpolate it in the same way. Parcae's models, trained with a loop count sampled around a mean of 8 and backpropagated through 4 of them, and run with 8, are worth 1.31–1.46 dense parameters per parameter at 140M–770M (Prairie et al., 2026), against a ladder trained on the same tokens per parameter, which understates the value. Ouro's models looped 4 times are worth 1.23 and 1.36 at 134M and 374M (about 150 and 50 tokens per parameter) and more than 1.75 at 778M, beyond its largest non-looped model, but less than their parameters at 53M despite about 380 tokens per parameter, measured by mean benchmark accuracy read off a plot (Zhu et al., 2025, App. D.1). The two 12-layer looped models of Saunshi et al. (2025), one looped twice as a whole and one with its middle four layers looped 4 times, are worth 1.26 and 1.36, and their shallower blocks looped 3, 4 and 6 times 1.47–1.95, at about 750–1500 tokens per parameter. The law of Schwethelm et al. (2026) ($\varphi = 0.46$, 95% interval 0.41–0.53) implies 1.25, 1.45 and 1.56 for 2, 4 and 8 loops in their iso-depth layouts. These studies differ from ours in size, data and recipe, so the agreement is a consistency check, not a pooled estimate. They barely sample models below 40M trained on 20 or fewer tokens per parameter, where we find a model with a looped middle block worth at most about as many dense parameters as it has. That the value rises with tokens per parameter at equal parameters is a dependence that effective-parameter laws whose loop gain depends on the loop count (and, for mixtures of experts, on the sparsity) but not on the training tokens (Schwethelm et al., 2026; Chen et al., 2026a) do not capture; it agrees with the recommendation of Prairie et al. (2026) to increase looping and data in tandem, and with the finding of Chen et al. (2026b) that with repeated data the best loop count grows with compute.

Where the gain lands, and what the seeds measure. Per token, the gain on web text is a small, broad shift: about half of the tokens improve, and the gain is not concentrated on hard tokens beyond what two dense seeds differ by *(exploratory)*. On math and code text the gain is larger, on average about 3 and 7 times the gain on the main set (1.5–7 and 4–11 across cells), but the seed-to-seed spread on those sets is 10–100 times larger as well. Supplementary experiment S7 *(pre-specified)* showed that the code set is even more sensitive to the data order, which our seeds share, than to the initialisation (the math set is not): at 20M, training the same initialisation on three disjoint parts of the data stream moves the main-set loss by only 0.003–0.010 nats, 0.1–0.5 times the spread between initialisation seeds (the plan had predicted one to three times), but moves the code-set loss by 0.21–0.69 nats, up to 7 times that spread, and changes the sign of the looped-minus-dense gap there. Of the plan's four predictions only this one, a code-set spread above 0.1, held: on the main set the data-order spread was smaller than predicted relative to the initialisation spread (and, for the dense model, below the predicted 0.005–0.02 nats), and the looped-minus-dense gap did not vary less than the losses. Both the initialisation and the documents a run happens to see move its out-of-distribution loss strongly at this size, so we draw no conclusions from differences on those sets.

D Learning rates (S1) and equal compute at the largest budgets (S4)

S1 re-tuned the dense model and the middle block looped 4 times at 10M, 20M and 40M: 0.5 to 1.41 times the dense rate, plus 0.35 times where the best rate lay at the lowest one tried, at $10N$–$40N$ (at 10M also $80N$ and $160N$), with one seed (two at 20M). Table 11 gives the gap between the middle block looped 4 times and the dense model at the shared rate (Δ_1) and with each model at its best rate ($\Delta^\star$), and $h = \Delta_1 - \Delta^\star$, the bias of the shared rate against the looped model. The pre-specified readings: the trend with size holds (at $40N$ the improvement from 10M to 40M is 0.050 against 0.065 at the shared rate, above the threshold of half of it), the trend with data holds and grows (10M from $10N$ to $160N$: 0.141 against 0.119), the equal-compute verdict was unchanged at the time (its 10M comparison at $160N$, extrapolated then, was later measured in S4: -0.011, a looped advantage, and $+0.005$ with a second seed; Section 4.4), and the bias exceeds the 0.01 tolerance at 10M only.

On the trunk checkpoints at $32N$ (the plan's exploratory reading Q5), the range of the main-set loss over 4–16 inference loops is 0.30, 0.36, 1.02, 3.39 and 4.10 nats at 0.35, 0.5, 0.71, 1 and 1.41 times the dense rate at 10M, 0.53–3.42 at 20M and 0.77–4.81 at 40M (0.5 to 1.41 times; the largest range at 40M is at the dense rate): lower rates make the iteration closer to convergent, but at every rate the better of 8 and 16 loops is worse than the 4 loops of training, by 0.07–3.0 nats.

Table 11: S1: looped (middle block, $r = 4$) minus dense loss at the shared rate and at each model's best rate (seed 42; 20M: mean of seeds 42 and 43). †The lowest rate tested.

Size, budget	Δ_1, shared rate	$\Delta^\star$, best rates	h	Best rate, dense / looped
10M, $10N$	+0.082	+0.082	0.000	1 / 0.71
10M, $20N$	+0.044	+0.026	+0.018	1 / 0.5
10M, $40N$	+0.015	−0.005	+0.020	1 / 0.5
10M, $80N$	−0.010	−0.035	+0.024	1 / 0.35†
10M, $160N$	−0.037	−0.059	+0.022	1 / 0.35†
20M, $10N$	+0.028	+0.028	0.000	1 / 1
20M, $20N$	−0.005	−0.005	0.000	1 / 1
20M, $40N$	−0.031	−0.031	0.000	1 / 1
40M, $10N$	−0.008	−0.008	0.000	1 / 1
40M, $20N$	−0.032	−0.036	+0.004	1 / 0.71
40M, $40N$	−0.050	−0.055	+0.005	0.71 / 0.71

Extrapolated equal-compute comparisons. Two extrapolations bracket a dense loss beyond the measured curve: in a backtest on the dense curves before S4 that drops the last budgets of every curve, extending the last segment predicts a dense loss 0.023–0.036 nats too low one doubling out (it flatters the dense model), and a power law fitted to the whole curve one 0.006–0.027 too high (it flatters looping). After S4, 62 equal-compute comparisons still need an extrapolated dense loss: 43 at 40M–160M, and 19 at 10M–20M, almost all for the third seed, whose dense curve ends at $80N$. At 160M and $40N$ the bracket reaches from +0.004 to +0.043.

Equal compute at the largest budgets (S4). Table 12 gives the equal-compute comparisons at the largest looped budgets. The S4 plan had predicted gaps between −0.01 and +0.04, the dense model ahead in most and none below −0.02; the last two held, but four of the nine measured top-budget gaps per seed exceeded +0.04 (up to +0.054; Table 12 gives the seed means of the four middle-block cells, and the ninth gap is that of the 10M whole stack looped 4 times, +0.042, one seed).

Table 12: The equal-compute comparisons at the largest looped budgets (S4; looped minus dense loss on the main validation set, means over two seeds; negative: looped better). At equal compute the dense model is trained on 2.2–3.8 times the tokens.

	Training FLOPs	Looped − dense loss	
Looped cell (size, tokens per parameter)	vs dense	equal parameters and tokens	equal compute
Middle block, $r = 4$ (10M, $160N$)	2.2×	−0.033	+0.016
Middle block, $r = 8$ (10M, $160N$)	3.7×	−0.051	+0.034
Middle block, $r = 4$ (20M, $80N$)	2.2×	−0.051	+0.022
Middle block, $r = 8$ (20M, $80N$)	3.8×	−0.064	+0.052
Middle block, $r = 4$, 0.35× rate (10M, $160N$)	2.2×	—	−0.011, +0.005 (seeds)

E Depth or sharing (S10)

The plan was frozen on 2026-10-27, before any of its runs. Dense models with 18 layers and the looped model's number of parameters (Table 13) were trained with the recipe of the grid's runs of the same size: the same data in the same order, batch, warm-up and optimiser settings, one constant-rate trunk with cooldown branches ending at $10N$, $20N$ and $40N$ of that size (in tokens identical to the looped runs), the same evaluation sets, and seeds 42, 43 and 44. The embedding and output layers use the frozen rate of their width (2.19×10^{-3} at width 320, 1.53×10^{-3} at 448) and the hidden layers $(18/12)^{-1/2}$ times it, as the code does for any depth; seed 42 was also trained at 0.71 and 1.41 times that rate. Training FLOPs per token

are 205M (dense, 8 layers), 456M (looped) and 239M (dense, 18 layers) at 20M, and 364M, 835M and 406M at 40M. None of the ten runs had a loss spike.

Table 13: S10: dense models with 18 layers (18×320, $N = 28.1$M; 18×448, $N = 51.2$M) against the middle block looped 4 times (8 unique layers, 20 executed; $N = 27.2$M and 51.0M), at the rule's learning rate. Losses and gaps are means over three seeds (gap = dense, 18 layers, minus looped loss, so positive means looped better, the opposite of the sign convention of Tables 8, 11 and 12; the gap per seed in parentheses); M is read off the 8-layer dense ruler; 95% intervals from resampling the three seeds, a rough guide with so few. At 20M the plan reports the results descriptively: M_{loop} is below 1 at $10N$ and near 1 at $20N$, and R is unstable (0.58, 10.1 and -0.40).

Size, budget	Dense, 18 layers	Looped	Gap (seeds)	M_{deep}	M_{loop}	R
20M, $10N$	3.654	3.671	-0.017 (-0.010, -0.016, -0.026)	0.84 [0.76, 0.98]	0.74 [0.67, 0.83]	—
20M, $20N$	3.465	3.456	$+0.008$ ($+0.017$, $+0.013$, -0.005)	0.92 [0.85, 1.01]	0.99 [0.92, 1.11]	—
20M, $40N$	3.335	3.307	$+0.028$ ($+0.037$, $+0.033$, $+0.015$)	0.94 [0.89, 1.00]	1.16 [1.09, 1.25]	—
40M, $10N$	3.397	3.394	$+0.003$ ($+0.004$, $+0.004$, -0.001)	1.10 [1.04, 1.14]	1.15 [1.08, 1.20]	0.67 [0.43, 1.03]
40M, $20N$	3.232	3.217	$+0.015$ ($+0.017$, $+0.018$, $+0.010$)	1.11 [1.09, 1.13]	1.26 [1.21, 1.29]	0.46 [0.38, 0.55]
40M, $40N$	3.119	3.090	$+0.029$ ($+0.030$, $+0.032$, $+0.026$)	1.11 [1.10, 1.12]	1.34 [1.31, 1.36]	0.36 [0.32, 0.41]

Readings fixed in advance. The primary reading is R at 40M averaged over the three budgets: at least 0.75 reads as "most of the advantage is a depth effect", 0.25–0.75 as "partly depth, partly sharing" and below 0.25 as "looping buys more than depth". It is 0.50 (95% interval 0.41–0.62). The plan would also have reported the two models as on par at 40M had the gap been within 0.01 nats at all three budgets; it is not ($+0.015$ and $+0.029$ at $20N$ and $40N$).

Learning rate. Seed 42 at 0.71, 1 and 1.41 times the rule's rate: at 40M 3.421, 3.396 and 3.382 at $10N$, 3.246, 3.231 and 3.225 at $20N$, and 3.126, 3.118 and 3.117 at $40N$; at 20M the rule's rate was the best at every budget. Because 1.41 times the rate beat the rule's rate by more than 0.01 nats at 40M and $10N$ (by 0.014), the plan asks for the readings also at the best rate tried, as in S1, the primary reading staying at the rule's rate. With each model at its best tested rate (seed 42; the looped model's best rates, from S1, are 1, 0.71 and 0.71 times the dense rate at $10N$, $20N$ and $40N$; both read off the 8-layer dense runs at their frozen rates, seed means, with `fit/supp3_best_lr.py`), the gap is -0.010, $+0.015$ and $+0.036$, M_{deep} 1.37, 1.17 and 1.12, M_{loop} 1.18, 1.33 and 1.41, and R 1.91, 0.56 and 0.34 (mean 0.94; for seed 42 at the rule's rate 0.58, 0.45 and 0.37, mean 0.47). The mean at the best rates would fall, by the bands the plan set for the primary reading, in "most of the advantage is a depth effect"; it is carried by $10N$, where $\log M_{\text{loop}}$ is smallest, and the primary reading stays at the rule's rate. The analysis script written before the results flagged this case but did not compute these readings; they come from a script written afterwards that implements the rule of the plan.

Predictions. The plan predicted R at 40M between 0.25 and 0.75 (held: 0.50); the 18-layer dense model better than the 8-layer one of the same size by 0.01–0.03 nats at 40M (0.007, 0.016 and 0.020 at $10N$, $20N$ and $40N$: held at two of the three budgets; at 20M it is worse by 0.014, 0.008 and 0.003); and the rule's rate within 0.01 nats of the best of the three (held at five of the six budgets).

Equal compute *(exploratory)*. Read as in Section 4.4, without extrapolation: the looped model is behind the 18-layer dense model of equal training compute by 0.166 nats at 40M and $10N$ (compute ratio 2.05) and by 0.193 and 0.112 at 20M and $10N$ and $20N$ (ratio 1.91).

F Deployment measurements

Inference was timed on one RTX 4090 (all other GPUs idle) in bf16, with the decoding step in a CUDA graph, one KV-cache slot per executed layer, random weights, a 512-token prompt and 512 generated tokens, at batch 1 and 32. Table 14 gives the ratios at equal loss for the cells with full backpropagation that were

timed (a middle block looped 2, 4 or 8 times, a whole stack looped 4 or 8 times) at 20M–160M, against the dense model that reaches the same loss at $40N$ (interpolated between dense sizes); 10M is left out because three of its five cells lie below the smallest dense model.

Table 14: S3: looped over dense, at equal loss. *An end of the range set by 160M, which lies beyond the largest dense model (extrapolated).

		Weight	KV cache	Decoding time	Peak memory	
Cell	Sizes	memory	per token	batch 1	batch 1	batch 32
Middle block, $r = 2$	20M–80M	0.93–1.03	1.43–1.51	1.45–1.49	0.99–1.15	1.21–1.24
Middle block, $r = 4$	20M–160M	0.81*–0.96	2.20*–2.41	2.37*–2.47	0.98*–1.19	1.57–1.58
Middle block, $r = 8$	20M–80M	0.83–0.92	4.01–4.23	4.27–4.56	1.08–1.34	2.29–2.40
Whole stack, $r = 4$	20M–80M	1.03–1.11	4.02–4.26	4.00–4.11	1.18–1.84	2.32–2.41
Whole stack, $r = 8$	20M–80M	0.81–0.85	6.96–7.23	7.57–7.91	1.27–1.50	3.38–3.91

For KV sharing (S6), the first 262,144 tokens of the main validation set were decoded token by token in 256 windows of 1024, with one cache slot per executed layer (whose loss equals that of the ordinary forward pass to four decimals) and with one slot per unique layer, in which every loop reads the earlier tokens' keys and values as written at their last loop. The models are the 20M trunk checkpoints at $32N$ (seed 42); Figure 8b gives the loss differences, and the random-loop model run with 16 loops instead of 8 also shares at no cost. The dense 20M model of the same seed, evaluated at the same checkpoint on the same tokens (added after a review on 2026-10-28), has loss 3.470; the fixed-loop models' gains over it are 0.027 ($r = 8$), 0.019 ($r = 8$, truncated), 0.029 ($r = 4$) and 0.046 ($r = 4$ at half the rate), and the random-loop models are 0.020–0.026 behind it.

At batch 1 the larger KV cache cancels the saving in weights: the looped model needs 1.5–2.4% less peak memory at 80M–160M and more at 20M–40M (up to 1.2 times). In this range, loops save parameters but not latency, and at batch 1 little or no peak memory; the saving would grow where weights dominate memory, for example one sequence with a short context in a much larger model, which we did not test. Truncated backpropagation does not make the cache shareable here, unlike in Huang et al. (2026). By the plan's rule (an increase of at most 0.02 nats: sharing usable; up to 0.1: it degrades; above 0.1: it breaks), the random-loop models are usable, the fixed 8-loop model degrades (0.091) and the truncated 8-loop model (0.140) and the 4-loop models break (0.170; 0.100 at half the learning rate, at the boundary). Among the fixed-loop models, a lower learning rate thus loses less. The S6 plan had predicted that the fixed 8-loop model would break (a loss increase above 0.3) and the truncated one would lose less than 0.1; the increases were 0.09 and 0.14, while the random-loop models shared at no cost and the lower learning rate lost less, as predicted (two of four).

G The synthetic task

Data and training. Tokens are the values 0–96 and a few markers (start, separator, lookup, answer and padding); a problem is a start marker, a start value and a separator, followed by d steps of the form `LUT k ;`, with the table index k written as one of the value tokens 0–31, and the answer marker: $3d + 4$ tokens. Problems are drawn online. During the first half of training the largest step count grows linearly from 1 to 8; the second half draws uniformly from 1–8. Models have width 256 and attention heads of dimension 64, and the looped models use the input injection and residual scaling of the language models. Training uses AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0), 400 warm-up steps and a cosine decay to a tenth of the peak rate, 512 problems per step, 32,000 steps. The two deepest conditions of S5 and S9 accumulated gradients over two half-batches to fit in memory (the same loss and update). Probes are linear maps from the state after each loop (or layer) to the 97 values, trained on 4,000 problems with 8 steps and tested on 1,000, at the table-index, separator and next-operation positions of each step, keeping the best.

Learning rates. The sweep (seed 1; 16,000 steps, half the formal length; 2.5×10^{-4}, 5×10^{-4} and 10^{-3}, extended at an edge) chose by mean in-range accuracy at the condition's training loop count: dense 8 and 20

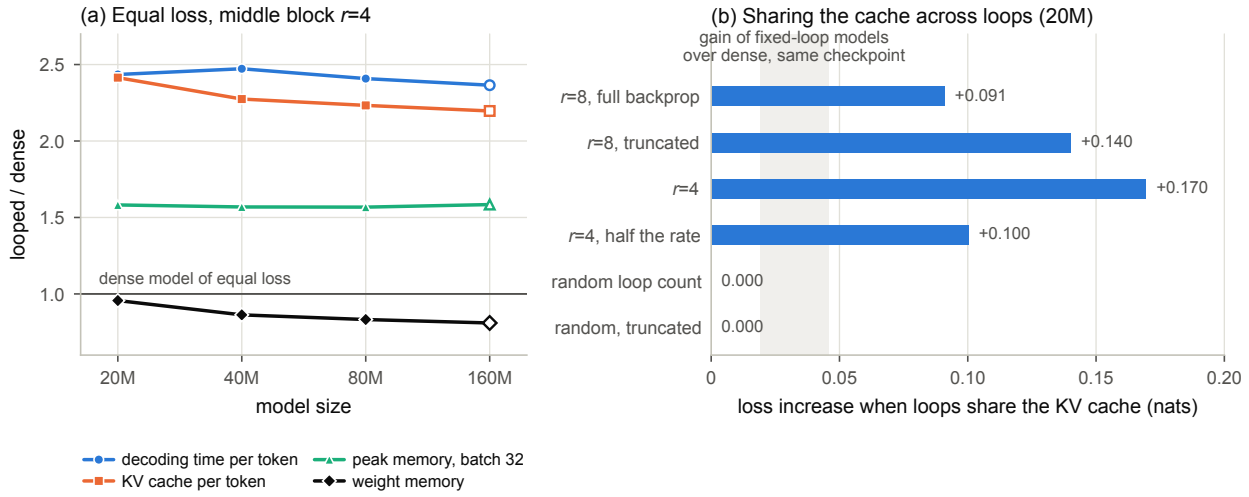


Figure 8: Deployment cost. **(a)** A middle block looped 4 times against the dense model of equal loss at $40N$: ratios of decoding time per token (batch 1), KV cache per token, peak memory at batch 32 (1k context) and weight memory (open markers: 160M lies beyond the largest dense model). **(b)** Sharing the KV cache across loops after training (20M, one cache slot per unique layer, written at a position's last loop): the loss increase on the main set; the shaded band is the range of the fixed-loop models' gains over the dense model of the same size, at the same checkpoint and on the same tokens (0.019–0.046).

layers 10^{-3}, dense 36 layers 5×10^{-4}, looped $r = 2$ and $r = 8$ 2.5×10^{-4}, looped $r = 4$, random loop count, loops = steps, the S9 model and the chain-of-thought models 1.25×10^{-4}, the pause model 5×10^{-4}. These are the rates of the embedding and output layers; the hidden layers use $(L/12)^{-1/2}$ times them, with L the number of unique layers (1.22, 0.77 and 0.58 for 8, 20 and 36), so the hidden layers of the 36-layer dense model train at about the rate of the 2- and 8-loop models. The criterion saturates at low rates (the loops = steps model scored 0.9999 at three rates and the chain-of-thought models 1.000 at all four); such ties went to the lowest rate by implementation, a rule the protocol had not specified, and the 4-loop, random-loop and S9 models were best at the lowest rate tried. Extrapolation is sensitive to this choice: in the sweep, the accuracy of the loops = steps model on 9 to 14 steps fell from 1.00 to 0.91 at 2.5×10^{-4} but to 0.05 at 1.25×10^{-4}. The 20-layer dense model reached 0.98 in range in its sweep run at 10^{-3} but 0.91 (0.85–0.99 across seeds) in the formal runs at that rate, so it may be under-tuned for the longer schedule.

Probes. The *frontier* is the first loop at which each value becomes decodable (probe accuracy ≥ 0.5), and *persistence* is the lowest probe accuracy a value reaches in the loops after that, averaged over the values (Figure 7). In 5 of the 10 seeds of the 20- and 36-layer dense models no intermediate value after the first appears at the step positions: four show only x_1 (probe accuracy 0.50–0.74) and, at the last layer, the answer, and one shows nothing. Measured by the share of values that first become decodable at their own loop, a reading of the design's diagonal share fixed after the results, the loops = steps and 8-loop models are diagonal (0.95 and 0.975) and the others are not (0.125); read as the share decoded most accurately (within 0.01 of the best) at their own loop, they are 0.625 and 0.425.

S2. Its plan identified the conditions with 4 and 8 loops, the random loop count and loops = steps as extrapolating and left out $r = 2$ on the strength of a number corrected later (Appendix H). The plan's readings: R1, the share of runs that learn the training range (accuracy at least 0.99), is 2 of 12 at 10^{-3} and 8 of 12 at 5×10^{-4}; R2, per cell, reads "the learning rate changes the solution" when at least two runs learned the range and extended on average at least 2 steps less than the formal runs, "does not support it" when they extended within 1 step, and is inconclusive otherwise: 8 loops at 5×10^{-4} and loops = steps at both rates do not support it, the random loop count at 5×10^{-4} supports it (2 against 4.2 steps), and the other four cells are inconclusive. Probes of all checkpoints of the seed-1 sweep *(exploratory)* show the step-by-step frontier at rates up to 5×10^{-4} and not above, for 4 and 8 loops and the random loop count alike; S2's own

probes at 10^{-3} still show about one loop per step for loops = steps (all three seeds) and for 4 loops in one seed, 2.5–7 loops per step for 8 loops, and no frontier for the random loop count. At 10^{-3} the runs that did not learn the training range reached 0.49–0.98 in range; at 5×10^{-4} the runs that learned it extended about as far as the formal runs (8 loops: 2.7 steps beyond the range against 3.2; loops = steps: 4.0 against 4.8), except the random loop count (2 against 4.2).

Where the chain of thought breaks. As soon as the prompt holds more than 8 tables, the first wrong token of the 8-layer chain-of-thought model is one of the first three written ones in 97–98% of the problems, and even when the correct earlier tokens are given, the tokens after the first two are mostly wrong (accuracy 0.04–0.10, with isolated positions much higher) *(exploratory)*.

Stopping rules. Figure 9 shows the confidence rule against the chain length. The plan's selection rule, the highest accuracy over 9–16 steps on the selection set among rules that keep the in-range accuracy within 0.01 of the training loop count's, also admitted votes over loop counts and picked a vote for three of the five conditions; without votes it picks the 0.5% state rule for $r = 4$, the confidence rule at 0.99 for $r = 8$ and the random loop count, and at 0.9 for $r = 2$; for S9 it picks the 0.5% state rule with or without votes. Where it picked a vote, the vote runs all 32 loops and comes within 0.014 of the best fixed count at 12 steps without beating it.

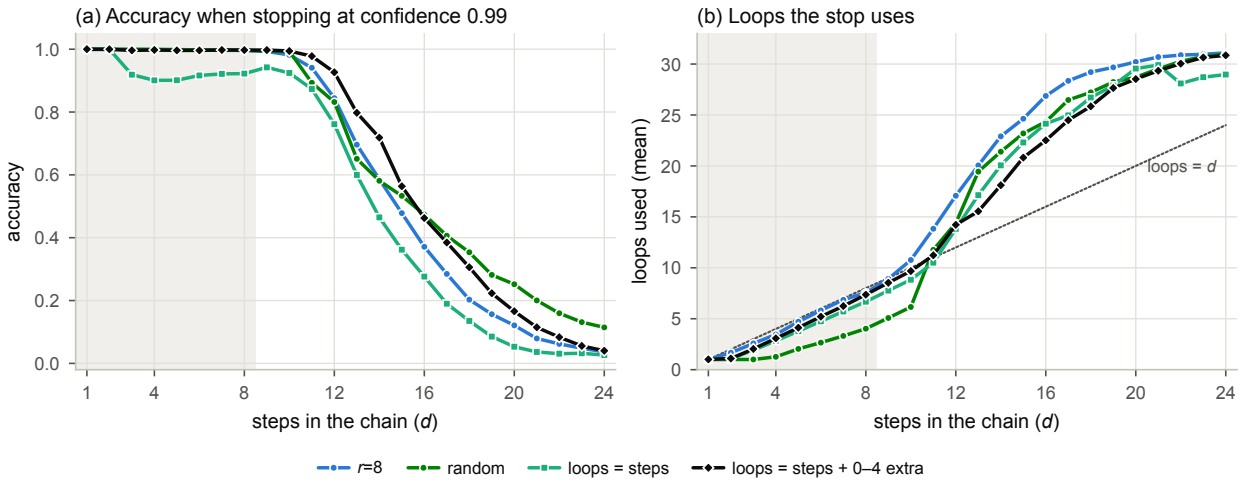


Figure 9: Stopping when the model's top probability reaches 0.99 (reporting set, mean over five seeds). **(a)** Accuracy. **(b)** Loops used: the stop runs few loops on short chains and many on long ones. The model trained with loops = steps loses part of the training range; trained with a few random extra loops, it keeps the training range and stops best of the four beyond it, up to 15 steps.

The pre-specified predictions. Table 15 lists the verdicts.

Supplementary predictions. S2: a high learning rate mainly prevents learning (Section 6.5). S5 held one of three (its optional fourth row, a looped model with intermediate supervision, was not run): the 8-layer chain-of-thought model was predicted to extend at least as far as the looped models (≥ 0.7 at 12 steps) and reached 0.014; pause tokens were predicted to add less than 0.1 and added nothing; the 36-layer chain-of-thought model was predicted to reach 0.8 at 12 steps and reached 0.017. S8 held two of five (Table 16): the prediction rule stayed within 1.5 times the training loop count in range, and votes did not beat the best fixed count; for $r = 4$ and $r = 8$, the prediction rule neither beat the state rule nor recovered 85% of the gain, and the confidence rule brought the loops = steps model to 0.76, above the predicted ceiling of 0.2. S9 held three of four; the one that failed is the frozen state rule's stop (0.52 at 12 steps against a predicted 0.6).

Pointer chasing. The design's second task, following a random function given in the prompt for k hops, was not learned by any of the four conditions tried on it under the recipe calibrated on the chain task: the

Table 15: The predictions of the frozen synthetic design and their verdicts.

Prediction	Verdict
P1: the largest solvable length rises from dense 8 layers to $r = 2, 4, 8$; $r = 8$ reaches 16	Holds in direction (4.2, 6.4, 8.0, 9.0); $r = 8$ does not reach 16.
P2: dense models of equal depth are not worse, and the gap is at most a quarter of the dense-8 to looped gap	Not supported as stated: they solve 1–1.6 fewer steps, within the tolerance for $r = 8$, not for $r = 4$.
P3a: fixed-loop models lose accuracy sharply at other loop counts	Half right: with fewer loops accuracy drops (the 8-loop model keeps 0.67 in range at 4 loops); with more loops the 4- and 8-loop models do not, in the direction that favours looping (with 16 loops the 8-loop model keeps 0.997 in range and solves 11.0 steps instead of 9).
P3b: the random-loop model does not improve with more loops beyond 8 steps	Fails: it gains on long chains (0.08 to 0.49 at 16 steps).
P3c: the loops = steps model with d loops beats itself with 8 loops beyond 8 steps, to at least 16	Holds (by 0.34–0.95 at every length from 10 to 16; the start at 10 steps and the 0.1 margin were fixed after the results, and at 9 steps the gain is 0.03).
P4: only the loops = steps model's probe table is diagonal	Half right: diagonal shares 0.95 (loops = steps) and 0.975 (8 loops), 0.125 for the others, by a definition fixed after the results; read as the loop at which a value is decoded most accurately (within 0.01 of the best), they are 0.625 and 0.425, and P4 would hold.
P5: the gaps are much larger than on text	Holds.

Table 16: S8, all rule families on the reporting set: accuracy at 12 / 16 steps (mean loops used at 12 steps). The rule the plan's selection criterion picks is in the last column.

Model	Training loops	Best fixed	State change $< 2\%$	Prediction unchanged	Confidence ≥ 0.99	Selected
$r = 2$	0.05 / 0.04	0.62 / 0.33	0.45 / 0.20 (29)	0.56 / 0.21 (6.3)	0.70 / 0.30 (8.3)	confidence ≥ 0.9
$r = 4$	0.04 / 0.02	0.83 / 0.29	0.65 / 0.21 (11)	0.48 / 0.09 (6.7)	0.81 / 0.26 (9.2)	vote, loops 8–32
$r = 8$	0.04 / 0.03	0.86 / 0.37	0.71 / 0.23 (17)	0.09 / 0.04 (5.5)	0.84 / 0.37 (17)	vote, loops 8–32
Random	0.77 / 0.08	0.83 / 0.48	0.73 / 0.37 (11)	0.28 / 0.11 (5.0)	0.83 / 0.47 (14)	vote, loops 8–32
Loops = steps	0.96 / 0.36	0.96 / 0.36	0.03 / 0.03 (25)	0.21 / 0.06 (8.8)	0.76 / 0.28 (14)	none eligible

36-layer dense and 8-loop models, two seeds each, reached 0.10–0.24 accuracy when trained to the end, and the random-loop and loops = steps models were stopped at 0.13 training accuracy; single-hop retrieval was not learned in that format either. It was dropped before any comparison on it.

H Deviations and corrections

Every item is dated in the deviation log in the repository.

- Before any data, the pre-registered training data (sample-100BT) were replaced by sample-10BT, which removed the $80N$ point of the 160M dense model, and the pre-registered data point, the mean of a cooldown's last three evaluations, by its final evaluation (Appendix A).
- Two power failures of the training machine cut CPU speed for a time; runs resumed from checkpoints and no result was affected.
- The synthetic task changed in three ways before its formal runs, all for learnability. Pilots of the 8-layer dense model, before any comparison between conditions, showed that a uniform mixture of 1–8-step problems was not learned (a curriculum was added) and that modular arithmetic chains were learned too slowly (fixed random lookup tables replaced them). A one-seed pilot of all conditions on the lookup task (2026-10-11), which already showed looped models extending beyond dense ones, then led to training of 32,000 steps for every condition (the looped conditions had not converged at 16,000) and to the learning-

rate protocol; the predictions, frozen on 2026-10-09, were not changed. Two further changes: three seeds were added to the first two (declared before their results), and the probe was widened from one to three positions per step after a diagnostic showed values stored at other positions.

- The learning-rate criterion of the synthetic task saturated, and ties went to the lowest rate by implementation (Appendix G).
- Checks of the results and of the paper after the runs (2026-10-14 to 2026-10-30), including two outside reviews (2026-10-14 and 2026-10-28), found errors that were corrected. Those that changed a verdict or a claim: the random-loop model's prediction (P3b) had first been judged by the longest solvable chain instead of the accuracy difference the design specifies; P3a had been misread; the readings of P3c and P4 had been fixed after the results without being marked as such; two of the five S8 predictions held, where one had been reported; the claim that confidence is the best stopping rule in every condition was withdrawn, and the statement that extra loops help only if the model can tell how many to run was limited to deployment; the equal-compute claim was limited to the measured range; the S6 cost of KV sharing had been compared with gains from other checkpoints (now matched: 2.2–7.4 times the gain); the deployment-FLOP inversion of H4 had dropped an intercept (the dense model stays optimal); the S3 memory figures had counted the KV cache twice; an extrapolation test of H3 had kept 160M in its training set, and an information criterion for H3, computable only as a proxy from the Huber loss, is no longer used as evidence; the set of loop counts behind the *More loops* columns of Table 3 was widened after the results; and Section 3.3 had said that a difference of 0.01 nats between runs that share a seed is resolvable, which holds only at 40M. The other corrections concerned rounded numbers, counts, citations, descriptions and results that had been computed but not reported. On 2026-10-30 the text was rewritten for readability, with no change to any result.
- The data selection of the equal-FLOP fit for H1 was corrected on 2026-10-07, before that verdict; the corrected fit is the one reported.
- The pre-registration's rule for the 10M rung of the ruler fit was first evaluated on 2026-10-28, after the paper was written; it does not apply (Appendix A).
- Two synthetic conditions of S5 and S9 ran out of GPU memory at the planned batch, having been tested only on a CPU before their plan was frozen; they were rerun with two accumulated half-batches.
- S10 (plan frozen 2026-10-27): its analysis script flagged that the learning-rate rule applied at 40M and $10N$ but did not compute the readings at the best rate; a script written after the results computes them (Appendix E). The script's intervals for the paired gap are not resampled, so the per-seed gaps are reported instead. The checkpoint size stated earlier (485 GB, measured on 2026-10-15) was not updated as S4–S10 added checkpoints; it is about 560 GB.
- One unexplained anomaly, observed on 2026-10-11: at 160M one looped seed is 0.2–0.6 nats worse than the other on the out-of-distribution sets at $40N$, while their main-set losses are identical.